

Process Algebras

Paolo Bettelini

Contents

I	Introduction and preliminaries	6
1	Program semantics	6
2	Mathematical preliminaries	6
2.1	Sets	6
2.2	Relations	6
2.3	Equivalence classes	7
2.4	Functions	8
3	Induction and inference systems	8
3.1	Mathematical induction	8
3.2	Inductively defined sets	9
3.3	Inference systems	9
4	Languages, grammars and automata	10
4.1	Strings and languages	10
4.2	Grammars	10
4.3	Labelled Transition Systems	11
4.4	Finite state automata	12
II	Operational semantics	14
5	The While language	14
5.1	Syntax	14
5.2	Numerals, states and expressions	14
6	Natural semantics	15
6.1	Rules	15
6.2	Induced meaning	16
7	Structural Operational Semantics	17
7.1	SOS rules	17
7.2	Semantic equivalence	18
7.3	Limitations of natural semantics	18
III	Orders and denotational semantics	19
8	Orders and functions over orders	19
8.1	Partial orders	19
8.2	Bounds, lattices and completeness	19

8.3	Monotone and Scott-continuous functions	20
8.4	Fixed points	20
9	Denotational semantics	21
9.1	Motivations	21
9.2	A review of the λ -calculus	21
9.3	Compositionality for While	23
9.4	The while construct as a fixed point	23
9.5	Information order on partial functions	23
9.6	CPOs and the fixed-point theorem	24
IV	Induction and coinduction	25
10	Examples of induction and coinduction	25
10.1	Mathematical induction and a non-example	25
10.2	Rule induction: finite traces	25
10.3	Rule coinduction: infinite traces	26
10.4	Finite and infinite lists	26
10.5	Convergence and divergence in the λ -calculus	27
11	Duality between induction and coinduction	27
12	Fixed-point theory for induction and coinduction	27
12.1	Complete lattices	27
12.2	Fixed-point theorem	28
12.3	From rules to functionals	29
12.4	Examples in fixed-point form	30
12.5	Bisimulation on lists	30
13	Other induction and coinduction principles	31
13.1	Mathematical induction as rule induction	31
13.2	Well-founded induction	32
13.3	Transfinite induction	32
13.4	Recursion and corecursion	32
13.5	Coinduction up-to	33
14	Bisimilarity as a fixed point	33
15	Constructive approximations of fixed points	33
15.1	Continuity and cocontinuity	33
15.2	Continuity and cocontinuity of rule systems	35
15.3	Iterative reading of the examples	36
15.4	Approximants of bisimilarity	36
15.5	Algorithms for bisimilarity	37
16	Interpretation by proof trees	38
17	Interpretation by games	38
17.1	The bisimulation game	39
V	Concurrency models and process operators	41
18	Concurrent programming	41
18.1	Communication	41
18.2	Why concurrent programming is difficult	41

19 Concurrent and reactive systems	43
19.1 Processes as an abstraction	43
20 Operational models for concurrency	43
20.1 Automata and LTSs	43
21 Regular expressions as a first process algebra	44
21.1 SOS rules	44
22 Operators for process modelling	45
22.1 Actions and basic processes	46
22.2 Sequentialization	46
22.3 Nondeterministic choice	47
22.3.1 Internal choice	47
22.3.2 External choice	47
22.4 Parallelism without communication	48
22.5 Milner parallelism and handshake communication	48
22.6 Merge with a synchronization function	49
22.7 Hoare parallelism	49
22.8 Synchronization algebra	49
22.9 Restriction and hiding	50
22.10 Relabelling	50
23 Infinite behaviours	51
23.1 Recursion	51
23.2 Replication	51
23.3 Iteration	51
23.4 Linking components	52
24 Operators with data	52
24.1 Value passing	52
24.2 Conditional	52
25 Advanced control operators	53
25.1 Pipelining	53
25.2 Disabling	53
25.3 Try/catch	53
VI Behavioural equivalences	55
26 Specifications, implementations and equivalences	55
27 Trace equivalence	56
28 Strong bisimulation	56
28.1 Operations on bisimulations	57
29 Bisimulation game	57
30 Simulation and similarity	58
31 Testing equivalences	58
31.1 Observations and success	59
32 Failures, futures and acceptance sets	59

33 Internal actions and weak equivalences	59
33.1 Weak transitions	60
33.2 Weak bisimulation	60
33.3 Branching bisimulation	60
33.4 The spectrum of equivalences	61
 VII CCS	 62
34 A Calculus for Communicating Systems	62
34.1 Names, co-names and actions	62
34.2 Syntax	62
35 SOS of CCS	62
35.1 Prefix and summation	63
35.2 Parallel composition	63
35.3 Restriction	63
35.4 Relabelling	63
35.5 Constants	63
36 The 50 prisoners problem	64
36.1 A first model	64
36.2 Mutual exclusion	64
36.3 Unknown initial state	65
37 Value-passing CCS	65
38 Buffers and bisimilarity	66
38.1 Finite-capacity buffers	66
38.2 A digression on “advanced proof methods”	66
39 Congruence of strong bisimilarity	67
40 Cells and FIFO buffers	67
40.1 Cells	67
40.2 FIFO with binary data	68
41 Weak bisimilarity in CCS	68
42 A communication protocol	68
 VIII Equational axiomatizations	 70
43 Why axiomatize an equivalence	70
44 Equational logic	70
44.1 Signatures and terms	70
45 Axioms for Basic CCS	71
46 Restriction and relabelling	71
47 Expansion theorem	72
48 Soundness for strong bisimilarity	72

49 Completeness for finite CCS	72
49.1 Standard form	72
50 Weak equivalences and observational congruence	73
51 Tau laws	74
51.1 Full standard form	74
 IX Modal logics and model checking	 77
52 Equivalence checking and model checking	77
52.1 Modal and temporal properties	77
53 Hennessy–Milner Logic	77
53.1 Syntax	77
53.2 Satisfaction relation	78
53.3 Negation	78
54 Model checking HML	79
55 HML and strong bisimilarity	80
56 Modal depth	81
57 Temporal properties and recursion	82
57.1 Inevitability and possibility	82
57.2 Recursive formulae	82
58 Set-theoretic semantics of HML	83
59 Functionals and fixed-point semantics	84
60 Examples of recursive properties	85
60.1 Reaching a state without an action	85
60.2 An invariant capability	85
60.3 Reaching a state that can perform b	85
60.4 Always being able to continue with b	85
61 A selection of temporal properties	85
62 Nested and mutually recursive formulae	86
63 Bisimilarity as a greatest fixed point	87
64 Main references	88

Part I

Introduction and preliminaries

1 Program semantics

Each programming language comes equipped with a *syntax* and a *semantics*. The syntax defines the legal programs, usually by means of a grammar, while the semantics defines their meaning, behaviour, and errors. Formal semantics is useful for language design and implementation, and in particular for reasoning about program or model correctness, equivalence, and refinement.

Esempio Motivation: ambiguity without formal semantics

Consider a C-like language and assume that initially $x = 1$. Expressions such as

$$y = x + + + x + +, \quad z = x + + - x + +$$

cannot be assigned an unambiguous meaning without fixing the evaluation order and the treatment of side effects. Similarly, consider the recursive definition

$$g(x) = g(x - 1).$$

The meaning of an expression involving $g(42)$ depends on how the language treats divergence and argument evaluation.

The situation becomes even more critical in the presence of concurrency. Besides the choices made by the compiler, we have internal nondeterminism caused by the parallel evaluation of programs that may cooperate on common tasks while competing for resources such as CPU, memory, and data.

2 Mathematical preliminaries

2.1 Sets

Let A, B be sets. Recall the operations

$$A \cup B, \quad A \cap B, \quad A \setminus B, \quad A \times B, \quad \mathcal{P}(A).$$

We write $A \subseteq B$ if every element of A belongs to B , and $A \subsetneq B$ if in addition $A \neq B$.

For a family $\{A_i\}_{i \in I}$ we can consider arbitrary unions and intersections

$$\bigcup_{i \in I} A_i, \quad \bigcap_{i \in I} A_i.$$

2.2 Relations

Definizione Binary relation

A *binary relation* between two sets A, B is a subset

$$R \subseteq A \times B.$$

We write aRb instead of $(a, b) \in R$.

On a set A , the identity relation is

$$\text{Id}_A = \{(a, a) \mid a \in A\}.$$

Given $R \subseteq A \times B$, its inverse relation is

$$R^{-1} = \{(b, a) \mid aRb\}.$$

Given $R \subseteq A \times B$ and $S \subseteq B \times C$, their composition is

$$S \circ R = \{(a, c) \mid \exists b \in B, aRb \wedge bSc\}.$$

For a relation $R \subseteq A \times A$, define

$$R^0 = \text{Id}_A, \quad R^{n+1} = R \circ R^n.$$

Its reflexive-transitive closure and transitive closure are respectively

$$R^* = \bigcup_{n \geq 0} R^n, \quad R^+ = \bigcup_{n \geq 1} R^n.$$

Thus aR^*b if and only if there exists a finite path, possibly of length zero, from a to b following edges of R .

Definizione Properties of a relation

Let $R \subseteq A \times A$. We say that R is

1. *reflexive* if aRa for every $a \in A$;
2. *irreflexive* if $a \not Ra$ for every $a \in A$;
3. *symmetric* if $aRb \implies bRa$;
4. *antisymmetric* if $aRb \wedge bRa \implies a = b$;
5. *transitive* if $aRb \wedge bRc \implies aRc$;
6. *total* if for every $a, b \in A$, either aRb or bRa .

Starting from a generic relation we can build closures that add the minimum number of pairs required to obtain properties such as reflexivity, symmetry, or transitivity.

Definizione Special relations

A relation $R \subseteq A \times A$ is

1. a *preorder* if it is reflexive and transitive;
2. a *partial order* if it is reflexive, antisymmetric, and transitive;
3. an *equivalence relation* if it is reflexive, symmetric, and transitive.

If R is a preorder, then

$$R \cap R^{-1}$$

is an equivalence relation, also called the symmetric kernel of the preorder.

2.3 Equivalence classes

Esempio Congruence modulo 3

On $\mathbb{Z}$, consider

$$n \sim_3 m \iff 3 \mid (n - m).$$

This is an equivalence relation with three classes, represented by the remainders 0, 1, 2.

Definizione Equivalence class and quotient

Let $\sim$ be an equivalence relation on A . The equivalence class of $a \in A$ is

$$[a]_{\sim} = \{b \in A \mid a \sim b\}.$$

The quotient set is

$$A/\sim = \{[a]_{\sim} \mid a \in A\}.$$

The equivalence classes partition A : two classes are either equal or disjoint, and their union is the whole set A .

2.4 Functions

A partial function $f: A \not\rightarrow B$ can be regarded as a relation $f \subseteq A \times B$ such that for every $a \in A$ there exists at most one $b \in B$ with afb . If such a b exists for every $a \in A$, then the function is total and we write $f: A \rightarrow B$.

Recall the usual notions of injectivity, surjectivity, and bijectivity. When the domain and codomain are ordered we can also study monotonicity and continuity, which will be fundamental for denotational semantics.

3 Induction and inference systems

3.1 Mathematical induction

Teorema Principle of mathematical induction

Let $P(n)$ be a property of natural numbers. If

1. $P(0)$ holds;
 2. for every $n \in \mathbb{N}$, $P(n) \implies P(n+1)$,
- then $P(n)$ holds for every $n \in \mathbb{N}$.

A variant allows us to assume in the inductive step that the property holds for all natural numbers less than or equal to n .

Esempio Sum of the first natural numbers

We want to prove

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

The basis is immediate. Assuming the formula for n , we have

$$\begin{aligned} 1 + \cdots + n + (n+1) &= \frac{n(n+1)}{2} + (n+1) \\ &= \frac{n(n+1) + 2(n+1)}{2} \\ &= \frac{(n+1)(n+2)}{2}. \end{aligned}$$

3.2 Inductively defined sets

Definizione Inductive definition

An inductive definition of a set contains

1. a set of base elements;
2. a set of construction rules;
3. the requirement to take the least set closed under those rules.

Natural numbers, finite lists, and finite trees are standard examples. For instance, fix a set A . The set of finite lists over A is the least set containing nil and satisfying

$$\ell \in L, a \in A \implies \langle a \rangle \cdot \ell \in L.$$

Teorema Structural induction

Let S be a set inductively defined by constructors. To prove a property P for every element of S , it is enough to prove that

1. P holds for all base elements;
2. every constructor preserves P , assuming it for its recursive arguments.

Esercizio Structural induction on lists

For a numerical list ℓ , prove

$$\text{sum}(\ell) \leq \max(\ell) \text{len}(\ell),$$

using a suitable convention for the empty list.

Soluzione

The base case is immediate. Consider a list $n :: \ell$ and assume the statement for ℓ . We distinguish two cases.

1. If $\max(\ell) \leq n$, then $\max(n :: \ell) = n$ and

$$\begin{aligned} \text{sum}(n :: \ell) &= n + \text{sum}(\ell) \\ &\leq n + \max(\ell) \text{len}(\ell) \\ &\leq n + n \text{len}(\ell) \\ &= n \text{len}(n :: \ell). \end{aligned}$$

2. If $n < \max(\ell)$, then $\max(n :: \ell) = \max(\ell)$ and

$$\begin{aligned} \text{sum}(n :: \ell) &\leq n + \max(\ell) \text{len}(\ell) \\ &\leq \max(\ell)(1 + \text{len}(\ell)). \end{aligned}$$

3.3 Inference systems

An inference rule has the form

$$\frac{\text{premises}}{\text{conclusion}}.$$

An axiom is a rule without premises. An inference system inductively defines the set of conclusions admitting a finite derivation.

Esempio Rational numbers as an inference system

We can describe rational numbers by rules constructing representations (p, q) with $q \neq 0$, and then quotient by the appropriate equivalence between representations. The important point is not the particular encoding, but the fact that a mathematical notion can be defined as the least set closed under an inference system.

Given a rule system R , associate with it an operator Φ_R on sets: $\Phi_R(X)$ contains precisely the conclusions of those rules whose premises belong to X . The operator is monotone and the inductively generated set can be obtained by the iteration

$$S_0 = \emptyset, \quad S_{n+1} = \Phi_R(S_n).$$

This construction will be formalized in detail when studying induction and coinduction.

Esempio Fibonacci by rules

Consider pairs (n, a) and the rules

$$\overline{(0, 0)}, \quad \overline{(1, 1)}, \quad \frac{(n+1, a) \quad (n, b)}{(n+2, a+b)}.$$

The successive iterations of Φ_R produce finite approximations of the graph of the Fibonacci function, and their union yields the complete function.

The first approximations are

$$\begin{aligned} S_1 &= \{(0, 0), (1, 1)\}, \\ S_2 &= S_1 \cup \{(2, 1)\}, \\ S_3 &= S_2 \cup \{(3, 2)\}, \\ S_4 &= S_3 \cup \{(4, 3)\}, \\ S_5 &= S_4 \cup \{(5, 5)\}, \\ S_6 &= S_5 \cup \{(6, 8)\}, \\ S_7 &= S_6 \cup \{(7, 13)\}. \end{aligned}$$

The chain keeps growing and its union is the graph of the Fibonacci function.

4 Languages, grammars and automata

4.1 Strings and languages

Let Γ be an alphabet. A string is a finite sequence of symbols from Γ , and the empty string is denoted by ε . We denote by Γ^* the set of all finite strings and call every subset $L \subseteq \Gamma^*$ a *language*.

4.2 Grammars

Definizione Grammar

A grammar is a quadruple

$$G = (N, T, P, S),$$

where N is the set of nonterminals, T is the set of terminals, P is the set of productions, and $S \in N$ is the start symbol.

A production $u \rightarrow v$ induces the derivation relation

$$s = lur, \quad t = lvr \quad \implies \quad s \Rightarrow t.$$

The relation $\Rightarrow^*$ is its reflexive-transitive closure. The generated language is

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}.$$

BNF is a compact notation for presenting grammars. Concrete syntax retains parentheses, precedences, and all the details required by parsing, whereas abstract syntax keeps only the semantically relevant structure.

For example, a concrete grammar encoding the usual precedence of arithmetic operators is

$$\begin{aligned} E &::= E + T \mid E - T \mid T, \\ T &::= T \cdot P \mid T/P \mid P, \\ P &::= N \mid (E), \\ N &::= DN \mid D, \\ D &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9. \end{aligned}$$

Thus $3 + 4 \cdot 5$ and $(3 + 4) \cdot 5$ have different parse trees without leaving precedence implicit.

Esempio Grammar for $\{a^n b^n c^n \mid n \geq 1\}$

Consider

$$T = \{a, b, c\}, \quad N = \{S, B\},$$

with start symbol S and productions

$$S \rightarrow aBSc \mid abc, \quad Ba \rightarrow aB, \quad Bb \rightarrow bb.$$

For instance,

$$\begin{aligned} S &\Rightarrow aBSc \Rightarrow aBaBScc \Rightarrow aBaBabccc \\ &\Rightarrow aaBBabccc \Rightarrow aaBaBbccc \Rightarrow aaaBBbccc \\ &\Rightarrow aaaBbbccc \Rightarrow aaabbbccc. \end{aligned}$$

Hence

$$L(G) = \{a^n b^n c^n \mid n \geq 1\}.$$

The usual pipeline is therefore

$$\text{source text} \longrightarrow \text{parsing} \longrightarrow \text{abstract syntax tree} \longrightarrow \text{execution/semantics}.$$

4.3 Labelled Transition Systems

Definizione Labelled Transition System

A *Labelled Transition System* is a quadruple

$$(Q, A, \rightarrow, q_0),$$

where Q is the set of states, A the set of actions, $\rightarrow \subseteq Q \times A \times Q$ the transition relation, and $q_0 \in Q$ the initial state. We write

$$q \xrightarrow{a} q'$$

for $(q, a, q') \in \rightarrow$.

A trace is a sequence of labels obtained by following a path in the LTS.

Esempio Vending machine

Consider an LTS for a vending machine whose states record, among other things, the initial configuration, the insertion of 25 or 50 cents, a choice, and the final outcome. Visible actions include

coin_in, cancel, return, choose, release.

A possible trace is

coin_in cancel return coin_in coin_in choose release $\dots$

This separates the internal state of the machine from the sequence of actions observed externally.

Esempio LTS semantics of arithmetic expressions

Consider arithmetic expressions built from numbers and binary operators. We can use rules of the form

$$\frac{m \circ n = k}{m \circ n \xrightarrow{\circ} k},$$

together with rules that first propagate reductions into the left subterm and then into the right subterm. A compound expression is therefore evaluated by a sequence of transitions until a value is reached.

For example,

$$\frac{4 + (7 \cdot 3)}{6 - 1} \xrightarrow{+} \frac{4 + 21}{6 - 1} \xrightarrow{+} \frac{25}{6 - 1} \xrightarrow{-} \frac{25}{5} \xrightarrow{/} 5.$$

The transitions are obtained by combining the rule that evaluates an operation on numbers with the rules that propagate a reduction into the left or right subterm.

4.4 Finite state automata

Definizione Finite state automaton

A finite state automaton is a quintuple

$$(Q, A, \rightarrow, q_0, F),$$

where the first four components form an LTS and $F \subseteq Q$ is the set of accepting states.

The transition relation is extended to words, and the recognized language is the set of words taking the initial state to an accepting state. Standard examples are automata recognizing strings with an even number of 1's or an odd number of 0's. Languages recognized by finite state automata are the regular languages. Regular languages are closed under intersection, union, difference, complement, reversal, concatenation, and star closure.

For example, over $\{0, 1\}$, let A_1 recognize the words containing an even number of 1's and let A_2 recognize the words containing an odd number of 0's. The product automaton has states recording both parities,

$$(e_0, e_1), (e_0, o_1), (o_0, e_1), (o_0, o_1).$$

By changing only the accepting states of this product we obtain automata for

$$L(A_1) \cup L(A_2), \quad L(A_1) \cap L(A_2), \quad L(A_1) \setminus L(A_2), \quad L(A_2) \setminus L(A_1).$$

This gives the usual product construction behind several closure properties of regular languages.

The Chomsky hierarchy can be summarized as follows:

Type	Restriction	Language	Abstract machine
0	unrestricted	recursively enumerable	Turing machine
1	$\alpha A \beta \rightarrow \alpha \gamma \beta$	context-sensitive	linear bounded automaton
2	$A \rightarrow \gamma$	context-free	nondeterministic pushdown automaton
3	$A \rightarrow a$ or $A \rightarrow aB$	regular	finite state automaton

where A, B are nonterminals, a is a terminal, and α, β, γ are strings of terminals and nonterminals.

Part II

Operational semantics

5 The While language

While is a very simple imperative programming language devised to illustrate different approaches to semantics. We study its BNF syntax, natural semantics or *big-step semantics*, Structural Operational Semantics or *small-step semantics*, and later its denotational semantics.

5.1 Syntax

We use the meta-variables

$$n \in \text{Num}, \quad x \in \text{Var}, \quad a \in \text{Aexp}, \quad b \in \text{Bexp}, \quad S \in \text{Stm}.$$

The syntax is

$$\begin{aligned} a &::= n \mid x \mid a_1 + a_2 \mid a_1 \cdot a_2 \mid a_1 - a_2, \\ b &::= \text{true} \mid \text{false} \mid a_1 = a_2 \mid a_1 \leq a_2 \mid \neg b \mid b_1 \wedge b_2, \\ S &::= x := a \mid \text{skip} \mid S_1; S_2 \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \mid \text{while } b \text{ do } S. \end{aligned}$$

Abstract syntax removes ambiguities of concrete syntax by fixing explicitly the tree structure of a program.

For example, the string

$$z := x; x := y; y := z$$

can be associated in different ways if the concrete grammar does not specify parentheses or associativity. Concrete syntax must therefore resolve ambiguities by precedence and parentheses, whereas abstract syntax assumes that the syntax tree has already been determined.

5.2 Numerals, states and expressions

Binary numerals can be described by

$$n ::= 0 \mid 1 \mid n0 \mid n1.$$

Define

$$\begin{aligned} \mathcal{N}[[0]] &= 0, & \mathcal{N}[[1]] &= 1, \\ \mathcal{N}[[n0]] &= 2\mathcal{N}[[n]], & \mathcal{N}[[n1]] &= 2\mathcal{N}[[n]] + 1. \end{aligned}$$

This also explains the usual joke: “there are 10 kinds of people, those who understand binary and those who do not”.

Definizione State

A state is a function

$$s: \text{Var} \rightarrow \mathbb{Z}.$$

Therefore

$$\text{State} = \text{Var} \rightarrow \mathbb{Z}.$$

For instance, we write

$$[x \mapsto 5, y \mapsto 7, z \mapsto 0]$$

to denote a state on the relevant variable names. Updating a state is defined by

$$(s[y \mapsto v])(x) = \begin{cases} v & x = y, \\ s(x) & x \neq y. \end{cases}$$

The semantics of arithmetic expressions is a function

$$\mathcal{A}: \text{Aexp} \rightarrow (\text{State} \rightarrow \mathbb{Z})$$

defined compositionally by

$$\begin{aligned} \mathcal{A}[\![n]\!]s &= \mathcal{N}[\![n]\!], \\ \mathcal{A}[\![x]\!]s &= s(x), \\ \mathcal{A}[\![a_1 + a_2]\!]s &= \mathcal{A}[\![a_1]\!]s + \mathcal{A}[\![a_2]\!]s, \end{aligned}$$

and analogously for multiplication and subtraction.

The Boolean semantics

$$\mathcal{B}: \text{Bexp} \rightarrow (\text{State} \rightarrow \{\text{tt}, \text{ff}\})$$

is defined in the same way. For example,

$$\begin{aligned} \mathcal{B}[\![\text{false}]\!]s &= \text{ff}, \\ \mathcal{B}[\![a_1 = a_2]\!]s &= \begin{cases} \text{tt} & \mathcal{A}[\![a_1]\!]s = \mathcal{A}[\![a_2]\!]s, \\ \text{ff} & \text{otherwise,} \end{cases} \\ \mathcal{B}[\![a_1 \leq a_2]\!]s &= \begin{cases} \text{tt} & \mathcal{A}[\![a_1]\!]s \leq \mathcal{A}[\![a_2]\!]s, \\ \text{ff} & \text{otherwise,} \end{cases} \end{aligned}$$

with negation and conjunction given by the usual truth tables.

6 Natural semantics

Operational semantics abstracts from machine details and describes program execution directly by rules. In particular, details such as registers, actual addresses of variables, machine architecture, and implementation strategies are ignored. Arithmetic and Boolean expressions inspect the state, whereas statements can both inspect and modify it. Natural semantics is a *big-step semantics*: it relates an initial configuration directly to its final result.

We write

$$\langle S, s \rangle \rightarrow s'$$

if executing S in state s terminates in state s' .

6.1 Rules

Definizione Natural semantics of While

The main rules are

$$\begin{aligned} &\overline{\langle x := a, s \rangle \rightarrow s[x \mapsto \mathcal{A}[\![a]\!]s]}, & \overline{\langle \text{skip}, s \rangle \rightarrow s}, \\ &\frac{\langle S_1, s \rangle \rightarrow s' \quad \langle S_2, s' \rangle \rightarrow s''}{\langle S_1; S_2, s \rangle \rightarrow s''}. \end{aligned}$$

For the conditional we have

$$\frac{\mathcal{B}[\![b]\!]s = \text{tt} \quad \langle S_1, s \rangle \rightarrow s'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'},$$

$$\frac{\mathcal{B}[b]s = \text{ff} \quad \langle S_2, s \rangle \rightarrow s'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'}.$$

Finally,

$$\frac{\mathcal{B}[b]s = \text{ff}}{\langle \text{while } b \text{ do } S, s \rangle \rightarrow s}$$

and

$$\frac{\mathcal{B}[b]s = \text{tt} \quad \langle S, s \rangle \rightarrow s' \quad \langle \text{while } b \text{ do } S, s' \rangle \rightarrow s''}{\langle \text{while } b \text{ do } S, s \rangle \rightarrow s''}.$$

Esempio Swapping two variables

Consider

$$(z := x; x := y); y := z$$

in the state

$$[x \mapsto 5, y \mapsto 7, z \mapsto 0].$$

A natural-semantics derivation shows that the final state satisfies $x = 7$, $y = 5$, and $z = 5$.

Esercizio Factorial

Determine the derivation, starting from the state $[x \mapsto 3]$, of the program

$$y := 1; \text{while } \neg(x = 1) \text{ do } (y := y \cdot x; x := x - 1).$$

Then repeat the same exercise using SOS.

The rule for while directly expresses its unfolding. In particular,

$$\text{while } b \text{ do } S$$

has the same big-step behaviour as

$$\text{if } b \text{ then } (S; \text{while } b \text{ do } S) \text{ else skip}.$$

Teorema Determinism of natural semantics

If

$$\langle S, s \rangle \rightarrow s_1 \quad \text{and} \quad \langle S, s \rangle \rightarrow s_2,$$

then $s_1 = s_2$.

The proof is by induction on the height of the derivation tree, rather than simply by structural induction on S : the rule for while contains the same statement again in its recursive premise.

6.2 Induced meaning

Natural semantics induces

$$\mathcal{S}_{ns}: \text{Stm} \rightarrow (\text{State} \nrightarrow \text{State})$$

defined by

$$\mathcal{S}_{ns}[S]s = \begin{cases} s' & \langle S, s \rangle \rightarrow s', \\ \text{undef} & \text{otherwise.} \end{cases}$$

It is a partial function because programs such as

$$\text{while true do skip}$$

do not terminate.

7 Structural Operational Semantics

SOS is a *small-step semantics*: it describes every individual computation step. A transition has the form

$$\langle S, s \rangle \Rightarrow \gamma,$$

where γ is either an intermediate configuration $\langle S', s' \rangle$ or a final state s' .

7.1 SOS rules

Definizione SOS of While

Assignments and skip terminate in one step:

$$\overline{\langle x := a, s \rangle \Rightarrow s[x \mapsto \mathcal{A}[a]s]}, \quad \overline{\langle \text{skip}, s \rangle \Rightarrow s}.$$

For sequential composition:

$$\frac{\langle S_1, s \rangle \Rightarrow \langle S'_1, s' \rangle}{\langle S_1; S_2, s \rangle \Rightarrow \langle S'_1; S_2, s' \rangle},$$

$$\frac{\langle S_1, s \rangle \Rightarrow s'}{\langle S_1; S_2, s \rangle \Rightarrow \langle S_2, s' \rangle}.$$

The conditional selects in one step the branch determined by the value of b :

$$\frac{\mathcal{B}[b]s = \text{tt}}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \Rightarrow \langle S_1, s \rangle},$$

$$\frac{\mathcal{B}[b]s = \text{ff}}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \Rightarrow \langle S_2, s \rangle}.$$

The while statement is unfolded by

$$\overline{\langle \text{while } b \text{ do } S, s \rangle \Rightarrow \langle \text{if } b \text{ then } (S; \text{while } b \text{ do } S) \text{ else skip}, s \rangle}.$$

A computation is therefore a finite or infinite sequence of SOS steps.

Teorema Determinism of SOS

From the same configuration, at most one SOS transition is applicable and its result is uniquely determined.

The following observation is useful when reasoning about sequential composition: a finite derivation of $S_1; S_2$ decomposes into a derivation that terminates S_1 , followed by a derivation of S_2 . The proof is by induction on the length of the computation.

More precisely, if

$$\langle S_1; S_2, s \rangle \Rightarrow^k s'',$$

then there exist s' and k_1, k_2 such that

$$k = k_1 + k_2, \quad \langle S_1, s \rangle \Rightarrow^{k_1} s', \quad \langle S_2, s' \rangle \Rightarrow^{k_2} s''.$$

Strong induction on the length is natural here: prove the case 0, then pass from all derivations of length at most k to derivations of length $k + 1$.

7.2 Semantic equivalence

Two statements are semantically equivalent under SOS if they have the same possible terminal states and also agree on the presence of infinite computations. The induced function is

$$\mathcal{S}_{sos}[[S]]s = \begin{cases} s' & \langle S, s \rangle \Rightarrow^* s', \\ \text{undef} & \text{otherwise.} \end{cases}$$

Teorema Equivalence of natural semantics and SOS

For every statement S and state s ,

$$\mathcal{S}_{ns}[[S]]s = \mathcal{S}_{sos}[[S]]s.$$

Proof

In one direction, prove

$$\langle S, s \rangle \rightarrow s' \implies \langle S, s \rangle \Rightarrow^* s'$$

by induction on the natural-semantics derivation tree.

For the converse, prove by induction on the number k of SOS steps that

$$\langle S, s \rangle \Rightarrow^k s' \implies \langle S, s \rangle \rightarrow s'.$$

The rules for sequential composition allow us to reconstruct exactly the premises of the corresponding big-step derivation.

7.3 Limitations of natural semantics

The differences between big-step and small-step semantics become important when the language is extended. Once their equivalence has been established for While, choosing between them is not only a matter of correctness but also of convenience. For some constructs one style can be much more natural, and the other may fail to expose information required by the application.

1. With an **abort** command, ordinary natural semantics does not distinguish divergence from abnormal blocking. SOS distinguishes an infinite computation from a finite stuck configuration. Big-step semantics can be repaired by introducing a distinguished error configuration.
2. With a nondeterministic choice S_1 or S_2 , natural semantics can hide a divergent branch if another branch terminates, whereas SOS preserves both behaviours.

For instance,

$$(\text{while true do skip}) \text{ or } (x := 2; x := x + 2).$$

3. With parallel composition, SOS can describe the interleaving of the component steps, whereas a relation directly connecting the initial state to the final state loses the required intermediate information.

Part III

Orders and denotational semantics

8 Orders and functions over orders

8.1 Partial orders

Definizione Partial order

Let S be a set and $\leq \subseteq S \times S$. The relation $\leq$ is a partial order if for every $a, b, c \in S$

1. $a \leq a$;
2. $a \leq b \wedge b \leq a \implies a = b$;
3. $a \leq b \wedge b \leq c \implies a \leq c$.

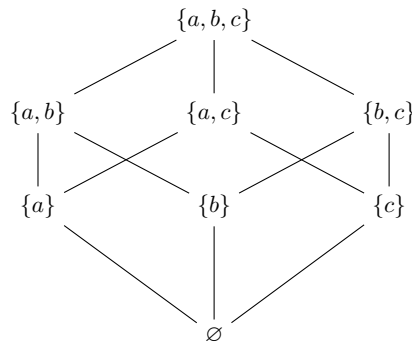
In this case $(S, \leq)$ is a *partially ordered set*, or simply a *poset*.

Examples are $(\mathbb{N}, \leq)$ and $(\mathcal{P}(X), \subseteq)$. An order is total if every pair of elements is comparable. A chain is a totally ordered subset.

Definizione Bottom and top

An element $\perp \in S$ is a *bottom* if $\perp \leq x$ for every $x \in S$. An element $\top \in S$ is a *top* if $x \leq \top$ for every $x \in S$.

8.2 Bounds, lattices and completeness



Given $A \subseteq S$, an upper bound u satisfies $a \leq u$ for every $a \in A$, whereas a lower bound l satisfies $l \leq a$ for every $a \in A$. The least upper bound, when it exists, is the join

$$\bigvee A,$$

while the greatest lower bound is the meet

$$\bigwedge A.$$

Definizione Directed set

A non-empty subset $D \subseteq S$ is *directed* if for every $x, y \in D$ there exists $z \in D$ such that

$$x \leq z, \quad y \leq z.$$

A poset is a *lattice* if every pair has a join and a meet. It is a *complete lattice* if every subset has a join and a meet. It is a *dcpo* if every directed subset has a join. Every complete lattice has

$$\perp = \bigvee \emptyset, \quad \top = \bigwedge \emptyset.$$

8.3 Monotone and Scott-continuous functions

Definizione Monotonicity

Let $(S, \leq_S)$ and $(T, \leq_T)$ be posets. A function $f: S \rightarrow T$ is monotone if

$$x \leq_S y \implies f(x) \leq_T f(y).$$

Definizione Scott continuity

Let S, T be dcpos. A monotone function $f: S \rightarrow T$ is Scott-continuous if for every directed set $D \subseteq S$

$$f\left(\bigvee D\right) = \bigvee f(D).$$

Proposition

Every Scott-continuous function is monotone.

Proof

If $a \leq b$, then $\{a, b\}$ is directed and $\bigvee \{a, b\} = b$. Therefore

$$f(b) = f\left(\bigvee \{a, b\}\right) = \bigvee \{f(a), f(b)\},$$

hence $f(a) \leq f(b)$.

Esercizio

Show that the converse is false: a monotone function need not be Scott-continuous.

8.4 Fixed points

Definizione Pre-fixed, post-fixed and fixed points

Let $f: L \rightarrow L$. An element $x \in L$ is

1. a *pre-fixed point* if $f(x) \leq x$;
2. a *post-fixed point* if $x \leq f(x)$;
3. a *fixed point* if $f(x) = x$.

Teorema Knaster–Tarski

Let L be a complete lattice and $f: L \rightarrow L$ monotone. Then the set of fixed points of f is a complete lattice. Moreover,

$$\text{gfp}(f) = \bigvee \{x \in L \mid x \leq f(x)\},$$

and

$$\text{lfp}(f) = \bigwedge \{x \in L \mid f(x) \leq x\}.$$

Proof Construction of the greatest fixed point

Put

$$g = \bigvee \{x \mid x \leq f(x)\}.$$

For every post-fixed point x , we have $x \leq g$, hence by monotonicity

$$f(x) \leq f(g).$$

Since $x \leq f(x)$, we obtain $x \leq f(g)$ for every post-fixed point, and therefore

$$g \leq f(g).$$

Applying monotonicity again,

$$f(g) \leq f(f(g)),$$

so $f(g)$ is itself post-fixed. By the definition of g , $f(g) \leq g$. Thus $f(g) = g$. The least fixed point case is dual.

Teorema Kleene fixed-point theorem

Let L be a dcpo with bottom $\perp$, and let $f: L \rightarrow L$ be Scott-continuous. Then

$$\text{lfp}(f) = \bigvee_{n \geq 0} f^n(\perp).$$

Proof

The chain

$$\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$$

is increasing by monotonicity. Put $p = \bigvee_n f^n(\perp)$. By continuity,

$$f(p) = f\left(\bigvee_n f^n(\perp)\right) = \bigvee_n f^{n+1}(\perp) = p.$$

If q is another fixed point, then by induction

$$f^n(\perp) \leq q$$

for every n , hence $p \leq q$.

9 Denotational semantics

9.1 Motivations

Denotational semantics associates a mathematical object with every programming-language construct. Its two central contributions are *compositionality* and a mathematical characterization of recursion through fixed points. The main ingredients are therefore functions, complete partial orders, and fixed-point theory.

9.2 A review of the λ -calculus

The λ -calculus is a minimal formal language for defining functions, function composition, and function evaluation. Its syntax is generated by variables, abstractions, and applications:

$$e ::= x \mid \lambda x. e \mid e_1 e_2.$$

We parenthesize compound applications explicitly whenever their association matters.

The fundamental computation rule is β -reduction

$$(\lambda x.d)e \longrightarrow d\{e/x\},$$

where substitution must avoid variable capture.

Esempio Simple λ -terms

$$\lambda x.x, \quad \lambda x.xx, \quad \lambda x.xz, \quad \lambda x.kz.$$

We can also use mixed notation such as $\lambda x.x + 1$ when arithmetic operators are already available in the metalanguage.

Church numerals are

$$0 \triangleq \lambda f.\lambda x.x, \quad 1 \triangleq \lambda f.\lambda x.fx, \quad 2 \triangleq \lambda f.\lambda x.f(fx),$$

and in general the numeral n applies f exactly n times. We can define

$$\text{succ} \triangleq \lambda n.\lambda f.\lambda x.f(nfx),$$

and, for instance,

$$\text{plus} \triangleq \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx),$$

or equivalently define addition in terms of `succ`. Then

$$\text{succ } 2 \longrightarrow^* 3, \quad \text{plus } 1 \ 2 \longrightarrow^* 3, \quad \text{plus } 4 \ 2 \longrightarrow^* 6.$$

Booleans can also be encoded:

$$\text{true} = \lambda x.\lambda y.x, \quad \text{false} = \lambda x.\lambda y.y.$$

Moreover,

$$\begin{aligned} \text{and} &\triangleq \lambda p.\lambda q.p \ q \ p, \\ \text{or} &\triangleq \lambda p.\lambda q.p \ p \ q, \\ \text{not} &\triangleq \lambda p.\lambda a.\lambda b.p \ b \ a, \\ \text{cond} &\triangleq \lambda p.\lambda a.\lambda b.p \ a \ b. \end{aligned}$$

If $e \longrightarrow^* \text{true}$, then

$$\text{cond } e \ y \ z \longrightarrow^* y,$$

and if $e \longrightarrow^* \text{false}$, the result is z .

Mixed notation emphasizes that ordinary functions can be represented in the same formalism:

$$\begin{aligned} \text{succ} &= \lambda x.x + 1, & \text{poly} &= \lambda x.x^2 - 3x + 1, \\ g &= \lambda x.\lambda y.x - y, & h &= \lambda y.\lambda x.x - y. \end{aligned}$$

For instance, $g \ 3 \ 4$ returns $3 - 4$, while $h \ 3 \ 4$ returns $4 - 3$. The main idea is that all objects of the λ -calculus are represented functionally: numbers, Booleans, states, arithmetic functions, and predicates can all be described in one formalism.

Self-application shows that evaluation need not terminate. Terms such as

$$(\lambda x.xx)(\lambda x.xx)$$

reduce to themselves. This possibility is essential for representing recursion and divergence.

9.3 Compositionality for While

The arithmetic and Boolean semantics defined earlier are already denotational. For statements we seek a function

$$\mathcal{S}_{ds} : \text{Stm} \rightarrow (\text{State} \not\rightarrow \text{State}).$$

For assignment and skip, define

$$\mathcal{S}_{ds} \llbracket x := a \rrbracket s = s[x \mapsto \mathcal{A} \llbracket a \rrbracket s], \quad \mathcal{S}_{ds} \llbracket \text{skip} \rrbracket = \text{Id}.$$

Sequential composition becomes function composition:

$$\mathcal{S}_{ds} \llbracket S_1; S_2 \rrbracket = \mathcal{S}_{ds} \llbracket S_2 \rrbracket \circ \mathcal{S}_{ds} \llbracket S_1 \rrbracket.$$

The conditional is

$$\mathcal{S}_{ds} \llbracket \text{if } b \text{ then } S_1 \text{ else } S_2 \rrbracket = \lambda s. \text{cond}(\mathcal{B} \llbracket b \rrbracket s, \mathcal{S}_{ds} \llbracket S_1 \rrbracket s, \mathcal{S}_{ds} \llbracket S_2 \rrbracket s).$$

9.4 The while construct as a fixed point

Let

$$W \triangleq \text{while } b \text{ do } S.$$

Its semantics must satisfy

$$\mathcal{S}_{ds} \llbracket W \rrbracket = F_{b,S}(\mathcal{S}_{ds} \llbracket W \rrbracket),$$

where

$$F_{b,S}(w) = \lambda s. \text{cond}(\mathcal{B} \llbracket b \rrbracket s, w(\mathcal{S}_{ds} \llbracket S \rrbracket s), s).$$

Thus we want to define

$$\mathcal{S}_{ds} \llbracket W \rrbracket = \text{fix}(F_{b,S}),$$

but we still have to determine which fixed point must be chosen.

Esempio A recursive equation can have several solutions

A definition such as

$$f(x) = \begin{cases} 1 & x = 0, \\ f(x+1) & x \neq 0 \end{cases}$$

does not determine a unique total function. Besides the least-defined solution, which returns 1 at 0 and is undefined elsewhere, there are solutions assigning an arbitrary constant value to points that never reach 0. Program semantics should select the solution containing only the information forced by the recursive definition.

9.5 Information order on partial functions

For partial functions $f, g : X \not\rightarrow Y$, define

$$f \sqsubseteq g \iff \forall x \in X, \quad f(x) \text{ defined} \implies f(x) = g(x).$$

Thus g contains at least as much information as f . The everywhere undefined function Ω is the bottom element.

For a recursive function we obtain the approximations

$$\Omega \sqsubseteq F(\Omega) \sqsubseteq F^2(\Omega) \sqsubseteq \dots$$

For factorial, the successive approximations define the result for progressively more inputs.

Esempio Approximations of the factorial while loop

If the body multiplies an accumulator y by x and decrements x , then the limit of the approximation chain is the function

$$f^\omega(x, y) = \begin{cases} (x, y) & x \leq 0, \\ (0, x!y) & x \geq 1. \end{cases}$$

This is the least fixed point of the functional associated with the loop.

9.6 CPOs and the fixed-point theorem

A CPO is a poset with bottom in which every increasing chain has a least upper bound. The set of partial functions ordered by information is a CPO.

The semantic functions used here are monotone and continuous with respect to this order, and composition preserves continuity. By Kleene's theorem,

$$\text{fix}(F) = \text{lfp}(F) = \bigvee_{n \geq 0} F^n(\perp).$$

This makes the semantics of while well defined: the domain of state functions is a CPO, the conditional is continuous, composition preserves continuity, and the loop functional is continuous.

Teorema Equivalence of SOS and denotational semantics

For every statement S ,

$$\mathcal{S}_{\text{sos}}[S] = \mathcal{S}_{\text{ds}}[S].$$

The proof uses a stronger invariant for a single SOS step and proceeds first by structural induction on the constructs and then by induction on the length of computations.

Part IV

Induction and coinduction

10 Examples of induction and coinduction

Induction and coinduction are dual principles. Induction describes objects constructed in finitely many steps and reasons about the *least* set closed under a collection of rules. Coinduction also describes potentially infinite behaviour and reasons about the *greatest* set compatible with suitable observations.

10.1 Mathematical induction and a non-example

Recall the principle of mathematical induction. A variant of the inductive step assumes the property for all natural numbers less than or equal to n . Other variants are possible, for instance by changing the basis.

Esempio A false proof by induction

Consider the statement “all men have eyes of the same colour”. The case of one man is trivial. Assume that every set of n men has only one eye colour, and consider $n + 1$ ordered men. The subsets in positions

$$\{1, \dots, n\} \quad \text{and} \quad \{2, \dots, n + 1\}$$

both contain n elements, so by the inductive hypothesis each subset would have a single eye colour. The argument then concludes that the two colours coincide because the subsets overlap. The mistake occurs in the step from $n = 1$ to $n = 2$: the two subsets are disjoint, so there is no common person forcing the two colours to coincide.

10.2 Rule induction: finite traces

Consider an LTS and say that a process is *stopped* if it cannot perform any transition. Write

$$P \downarrow$$

if P has a finite sequence of transitions leading to a stopped process. This property, also called *may termination*, is inductively defined by

$$\frac{P \text{ stopped}}{P \downarrow}, \quad \frac{P \xrightarrow{\mu} P' \quad P' \downarrow}{P \downarrow}.$$

A set inductively defined by rules admits three equivalent readings:

1. the set of elements having a finite, or more generally well-founded, proof tree;
2. the least set closed *forwards* under the rules;
3. the set obtained by an iterative construction starting from the empty set.

We will justify all three readings through fixed-point theory.

Definizione Forward closure

A set T is closed forwards under a system of rules if, whenever all premises of a rule belong to T , its conclusion also belongs to T .

For $\downarrow$, every forward-closed set must contain all stopped processes and, whenever it contains a derivative P' reachable from P , it must also contain P . The least set with this property is exactly the set of processes having a finite trace to a stopped state.

Esempio A function defined on terminating processes

Define partially

$$f(P) = \text{minimum length of a trace from } P \text{ to a stopped process.}$$

The domain of f is closed forwards under the rules for $\downarrow$: stopped processes are in the domain and, if $P \xrightarrow{\mu} P'$ with $P' \in \text{dom}(f)$, then $P \in \text{dom}(f)$ as well. By rule induction,

$$\{P \mid P \downarrow\} \subseteq \text{dom}(f).$$

10.3 Rule coinduction: infinite traces

Write

$$P \uparrow$$

if P can perform an infinite trace. The coinductive definition uses only the rule

$$\frac{P \xrightarrow{\mu} P' \quad P' \uparrow}{P \uparrow}.$$

There is no base case: a proof may be infinite.

The dual coinductive reading is:

1. proof trees need not be well founded;
2. we look for the greatest set closed *backwards*;
3. we start from the universe of all processes and progressively remove those that cannot satisfy the rule.

Definizione Backward closure

A set T is closed backwards under the rules if, for every $x \in T$, there exists at least one rule with conclusion x whose premises all belong to T .

Thus, to prove $P \uparrow$, we do not necessarily have to construct the whole infinite trace. It is enough to find a candidate set T containing P such that every process in T has at least one transition to another element of T .

Esempio Coinductive proof of divergence

If $P_1 \xrightarrow{a} P_2$ and $P_2 \xrightarrow{b} P_1$, take

$$T = \{P_1, P_2\}.$$

Every element of T has a transition to another element of T , so T is backward closed and

$$P_1 \uparrow, \quad P_2 \uparrow.$$

10.4 Finite and infinite lists

Fix a set A . Finite lists are inductively defined by

$$\frac{}{\text{nil} \in \text{finLists}}, \quad \frac{\ell \in \text{finLists} \quad a \in A}{\langle a \rangle \cdot \ell \in \text{finLists}}.$$

If the same observation pattern is interpreted coinductively, we obtain both finite and infinite lists. A coinductive object is observed through destructors: a list is nil, or it exposes a head a and a tail that is again a list.

10.5 Convergence and divergence in the λ -calculus

Under call-by-name, convergence $e \Downarrow e'$ can be inductively defined by

$$\frac{}{\lambda x.e \Downarrow \lambda x.e'},$$

$$\frac{e_1 \Downarrow \lambda x.e_0 \quad e_0\{e_2/x\} \Downarrow e'}{e_1 e_2 \Downarrow e'}.$$

Divergence $e \Uparrow$ is instead coinductive:

$$\frac{e_1 \Uparrow}{e_1 e_2 \Uparrow}, \quad \frac{e_1 \Downarrow \lambda x.e_0 \quad e_0\{e_2/x\} \Uparrow}{e_1 e_2 \Uparrow}.$$

The first system describes the least convergence relation compatible with the rules, whereas the second describes the greatest set of terms compatible with the observations of divergence.

11 Duality between induction and coinduction

Induction is driven by *constructors*: to belong to an inductively defined set, an object must be obtained from base cases by finitely many applications of the constructors. Coinduction is driven by *observers*: an object belongs to a coinductively defined set when it keeps producing admissible observations.

The main correspondences are:

Induction	Coinduction
least set	greatest set
forward closure	backward closure
least fixed point	greatest fixed point
pre-fixed points	post-fixed points
constructors	observers/destructors
congruences	bisimulations
substitutive relations	bisimulative relations
identity/syntax	bisimilarity/semantics
algebras	coalgebras
well-founded proofs	possibly non-well-founded proofs
strengthen the candidate	weaken/enlarge the candidate

For instance, a congruence is typically defined forwards: if two components are related, applying a constructor produces related objects again. A bisimulation is instead defined backwards from observations: every observation on one side must be matched by the other side.

The two techniques interact in process algebra: we want bisimilarity, obtained coinductively, to be a congruence with respect to the syntactic operators.

12 Fixed-point theory for induction and coinduction

12.1 Complete lattices

Consider a complete lattice $(L, \leq)$. Every set $S \subseteq L$ has both a meet and a join. For the fundamental example $L = \mathcal{P}(X)$, the order is inclusion and

$$\bigvee_i S_i = \bigcup_i S_i, \quad \bigwedge_i S_i = \bigcap_i S_i, \quad \perp = \emptyset, \quad \top = X.$$

Esempio Divisibility order

Consider the positive natural numbers ordered by

$$n \leq m \iff n \mid m.$$

For $S = \{4, 8, 16\}$, the lower bounds are $\{1, 2, 4\}$, the meet is 4, and 16 is both the maximum and the join. For $S = \{2, 3, 4\}$, the meet is 1, there is no minimum element of S , the upper bounds are the multiples of 12, the join is 12, and there is no maximum.

Esempio Pre-fixed points and fixed points

On the positive natural numbers, let $F(n)$ be the sum of the proper divisors of n , with the exception $F(1) = 1$. For example,

$$F(1) = 1, \quad F(2) = 1, \quad F(3) = 1, \quad F(4) = 3, \quad F(6) = 6.$$

With respect to divisibility, 1, 2, 3, 6 are pre-fixed points and 1, 6 are fixed points.

Esercizio

Determine whether $\mathbb{N}$ is a complete lattice and whether it is at least a lattice. Determine also which elements can be added to obtain a complete lattice.

Esercizio

Let F be monotone on a complete lattice. Show that the join of two post-fixed points is again post-fixed, generalize to an arbitrary family, and give the dual result for pre-fixed points.

12.2 Fixed-point theorem

For simplicity, first state the theorem on $\mathcal{P}(X)$.

Teorema Fixed-point theorem

Let

$$F: \mathcal{P}(X) \rightarrow \mathcal{P}(X)$$

be monotone. Then

$$\text{lfp}(F) = \bigcap \{T \subseteq X \mid F(T) \subseteq T\},$$

and

$$\text{gfp}(F) = \bigcup \{T \subseteq X \mid T \subseteq F(T)\}.$$

Thus the least fixed point is the meet of all pre-fixed points and the greatest fixed point is the join of all post-fixed points.

The fixed points, pre-fixed points, and post-fixed points themselves inherit complete-lattice structure.

Proof Greatest fixed point

Put

$$T = \bigcup \{S \subseteq X \mid S \subseteq F(S)\}.$$

For every post-fixed point S , $S \subseteq T$, hence by monotonicity

$$F(S) \subseteq F(T).$$

Since $S \subseteq F(S)$, we obtain $S \subseteq F(T)$. Taking the union over all post-fixed points gives

$$T \subseteq F(T).$$

Applying F and monotonicity,

$$F(T) \subseteq F(F(T)),$$

so $F(T)$ is post-fixed. Since T is the union of all post-fixed points,

$$F(T) \subseteq T.$$

Therefore $F(T) = T$, and T is the greatest fixed point.

Definizione Inductive and coinductive sets of a functional

Given a monotone endofunctional F on a complete lattice of sets, define

$$F_{\text{ind}} \triangleq \text{lfp}(F), \quad F_{\text{coind}} \triangleq \text{gfp}(F).$$

12.3 From rules to functionals

Definizione Ground rule

Let X be a set. A ground rule over X is a pair

$$(S, x), \quad S \subseteq X, \quad x \in X.$$

We write it as

$$\frac{x_1 \quad x_2 \quad \cdots}{x}, \quad S = \{x_1, x_2, \dots\}.$$

A rule $(\emptyset, x)$ is an axiom.

Rules with meta-variables are not ground, but they can be translated into ground rules by taking all valid instantiations.

Definizione Functional associated with a rule system

Given a set of ground rules R over X , define

$$\Phi_R(T) \triangleq \{x \in X \mid \exists (S, x) \in R, S \subseteq T\}.$$

Proposition

Φ_R is monotone.

Esercizio

Show that every monotone operator on $\mathcal{P}(X)$ can be represented as the functional associated with a suitable set of ground rules.

By the fixed-point theorem,

$$\text{lfp}(\Phi_R)$$

is the set inductively defined by the rules, while

$$\text{gfp}(\Phi_R)$$

is the set coinductively defined by them.

The condition

$$\Phi_R(T) \subseteq T$$

says exactly that T is forward closed: every axiom concludes an element of T , and every rule whose premises are in T has its conclusion in T . Thus rule induction is the principle

$$\Phi_R(T) \subseteq T \implies \text{lfp}(\Phi_R) \subseteq T.$$

Dually,

$$T \subseteq \Phi_R(T)$$

says that T is backward closed, and rule coinduction is

$$T \subseteq \Phi_R(T) \implies T \subseteq \text{gfp}(\Phi_R).$$

Esercizio

Suppose every rule in R has a non-empty set of premises. Show that

$$\text{lfp}(\Phi_R) = \emptyset.$$

12.4 Examples in fixed-point form

For finite traces, the ground rules are

$$R_{\downarrow} = \{(\emptyset, P) \mid P \text{ stopped}\} \cup \{(\{P'\}, P) \mid P \xrightarrow{\mu} P' \text{ for some } \mu\}.$$

Therefore

$$\Phi_{R_{\downarrow}}(T) = \{P \mid P \text{ is stopped, or } P \xrightarrow{\mu} P' \text{ with } P' \in T\}.$$

The processes having a finite trace form $\text{lfp}(\Phi_{R_{\downarrow}})$.

For infinite traces,

$$R_{\uparrow} = \{(\{P'\}, P) \mid P \xrightarrow{\mu} P'\},$$

and

$$\Phi_{R_{\uparrow}}(T) = \{P \mid \exists P' \in T, P \xrightarrow{\mu} P'\}.$$

The processes having an infinite trace form $\text{gfp}(\Phi_{R_{\uparrow}})$.

For finite lists,

$$F(T) = \{\text{nil}\} \cup \{\langle a \rangle \cdot \ell \mid a \in A, \ell \in T\}.$$

Hence

$$\text{finLists} = \text{lfp}(F).$$

List induction is precisely the pre-fixed-point principle applied to this functional.

For the λ -calculus, the convergence functional acts on relations of pairs of closed terms:

$$\begin{aligned} \Phi_{\downarrow}(T) = & \{(e, e') \mid e = e' = \lambda x.e'' \text{ for some } e''\} \\ & \cup \{(e, e') \mid e = e_1 e_2, \exists e_0 : (e_1, \lambda x.e_0) \in T, (e_0\{e_2/x\}, e') \in T\}. \end{aligned}$$

For divergence,

$$\Phi_{\uparrow}(T) = \{e_1 e_2 \mid e_1 \in T\} \cup \{e_1 e_2 \mid e_1 \Downarrow \lambda x.e_0, e_0\{e_2/x\} \in T\}.$$

12.5 Bisimulation on lists

Coinduction is not specific to concurrency. Consider finite or infinite lists and the LTS

$$\langle a \rangle \cdot s \xrightarrow{a} s.$$

Definizione List bisimulation

A relation R on lists is a list bisimulation if, for every $(s, t) \in R$,

1. $s = \text{nil} \implies t = \text{nil}$;
2. if $s = \langle a \rangle \cdot s'$, then there exists t' such that

$$t = \langle a \rangle \cdot t' \quad \text{and} \quad (s', t') \in R.$$

List bisimilarity is the union of all list bisimulations.

Proposition

For finite or infinite lists,

$$s = t \iff s \sim t.$$

Equality itself can be presented coinductively by the rules

$$\frac{}{\text{nil} = \text{nil}}, \quad \frac{s_1 = s_2 \quad a \in A}{\langle a \rangle \cdot s_1 = \langle a \rangle \cdot s_2}.$$

The inductive interpretation gives equality on finite lists; the coinductive interpretation also includes infinite lists, and its post-fixed points are precisely the list bisimulations.

Esempio Coinduction with map and iterate

Let $f: A \rightarrow A$ and define

$$\begin{aligned} \text{map } f \text{ nil} &= \text{nil}, \\ \text{map } f (\langle a \rangle \cdot s) &= \langle f(a) \rangle \cdot \text{map } f s, \\ \text{iterate } f a &= \langle a \rangle \cdot \text{iterate } f f(a). \end{aligned}$$

We want to prove

$$\text{map } f (\text{iterate } f a) = \text{iterate } f f(a).$$

Take

$$R = \{(\text{map } f (\text{iterate } f a), \text{iterate } f f(a)) \mid a \in A\}.$$

Both sides expose the head $f(a)$, and the resulting tails are again related by R . Thus R is a bisimulation and the result follows by coinduction.

13 Other induction and coinduction principles

13.1 Mathematical induction as rule induction

The rules

$$\frac{}{0}, \quad \frac{n}{n+1} \quad (n \geq 0)$$

inductively define the natural numbers. Thus rule induction coincides with mathematical induction.

The strong variant uses rules of the form

$$\frac{0, 1, \dots, n}{n+1},$$

or, in open form,

$$\frac{i \in S \text{ for every } i < j}{j \in S}.$$

13.2 Well-founded induction

Teorema Well-founded induction

Let R be a well-founded relation on X . To prove a property T for all elements of X , it is enough to prove for every $x \in X$ that

$$(\forall y, yRx \implies y \in T) \implies x \in T.$$

Mathematical induction and structural induction are special cases. A common example is induction on the lexicographic order of pairs of natural numbers.

The principle can be derived from rule induction by choosing, for every x , the rule

$$\frac{\{y \mid yRx\}}{x}.$$

If R is well founded, the inductively generated set is all of X .

13.3 Transfinite induction

Transfinite induction extends mathematical induction to ordinals. To prove a property T for all ordinals, show that

$$(\forall \beta < \alpha, \beta \in T) \implies \alpha \in T.$$

In practice, one distinguishes three cases:

1. $0 \in T$;
2. if $\alpha \in T$, then $\alpha + 1 \in T$;
3. if β is a limit ordinal and $\alpha \in T$ for every $\alpha < \beta$, then $\beta \in T$.

The ordinals form a proper class, so this formulation is not obtained directly from the set-based lattice theory developed here. In applications, however, ordinals can be used to rank elements of a set, inducing a well-founded relation; transfinite induction then becomes an instance of well-founded induction.

Other common induction principles include structural induction, induction on derivation trees, and induction on transitions.

13.4 Recursion and corecursion

A function defined by *recursion* typically has an inductive domain. Examples are

$$f(0) = 1, \quad f(n+1) = n f(n),$$

and structural recursion on λ -terms

$$\begin{aligned} f(x) &= 0, \\ f(\lambda x.e) &= 1 + f(e), \\ f(ee') &= f(e) + f(e'). \end{aligned}$$

General schemes of well-founded recursion exist; well-founded induction can be used to prove existence and uniqueness of the specified function.

A function defined by *corecursion* instead produces an element of a coinductive set. The equations specify the immediate observations of the produced object: for an infinite list, for example, they determine the head and how to obtain the tail. The functions `map` and `iterate` are natural examples. General corecursion schemes similarly provide existence and uniqueness, including mutually corecursive systems of equations.

13.5 Coinduction up-to

The coinduction principle can be strengthened by allowing the candidate to be closed using relations that are already known.

Teorema Coinduction up-to join

Let F be monotone on a complete lattice and let y be a post-fixed point, that is,

$$y \leq F(y).$$

Then

$$\text{gfp}(F) = \bigvee \{x \mid x \leq F(x \vee y)\}.$$

In particular,

$$x \leq F(x \vee y) \implies x \leq \text{gfp}(F).$$

A more general principle is coinduction *up-to gfp*. Let $\bullet: L \times L \rightarrow L$ be associative and assume:

1. if $x \leq F(x')$ and $y \leq F(y')$, then

$$x \bullet y \leq F(x' \bullet y');$$

2. for every post-fixed point x ,

$$x \leq x \bullet \text{gfp}(F), \quad x \leq \text{gfp}(F) \bullet x.$$

Then

$$x \leq F(\text{gfp}(F) \bullet x \bullet \text{gfp}(F)) \implies x \leq \text{gfp}(F).$$

14 Bisimilarity as a fixed point

Let Proc be the set of states of an LTS. Define

$$F_{\sim}: \mathcal{P}(\text{Proc} \times \text{Proc}) \rightarrow \mathcal{P}(\text{Proc} \times \text{Proc})$$

by putting $(P, Q) \in F_{\sim}(R)$ if, for every action μ ,

1. every $P \xrightarrow{\mu} P'$ is matched by some $Q \xrightarrow{\mu} Q'$ with $P'RQ'$;
2. symmetrically, every $Q \xrightarrow{\mu} Q'$ is matched by some $P \xrightarrow{\mu} P'$ with $P'RQ'$.

Proposition

1. $F_{\sim}$ is monotone;
2. R is a bisimulation if and only if

$$R \subseteq F_{\sim}(R);$$

3. bisimilarity is

$$\sim = \text{gfp}(F_{\sim}).$$

This formulation explains formally why bisimulation is a coinductive proof method.

15 Constructive approximations of fixed points

15.1 Continuity and cocontinuity

The fixed-point theorem above is existential and does not by itself give an iterative algorithm. To obtain fixed points constructively, introduce continuity and cocontinuity.

Definizione Continuity and cocontinuity

An endofunctional F on a complete lattice is

1. *continuous* if for every increasing chain $T_0 \leq T_1 \leq \dots$,

$$F\left(\bigvee_i T_i\right) = \bigvee_i F(T_i);$$

2. *cocontinuous* if for every decreasing chain $T_0 \geq T_1 \geq \dots$,

$$F\left(\bigwedge_i T_i\right) = \bigwedge_i F(T_i).$$

Esempio

Consider the integers extended with two points $-\omega$ and ω , ordered naturally, and define

$$F(n) = n + 1, \quad F(\omega) = \omega, \quad F(-\omega) = -\omega.$$

On increasing chains of positive integers and decreasing chains of negative integers, the operator preserves the relevant joins and meets.

Esercizio

Show that every continuous or cocontinuous function is monotone. For $x \geq y$, use an eventually constant chain such as $x, y, y, \dots$ in the appropriate direction.

Define the iterates

$$F^0(x) = x, \quad F^{n+1}(x) = F(F^n(x)),$$

and

$$F^{\cup\omega}(x) = \bigvee_{n \geq 0} F^n(x), \quad F^{\cap\omega}(x) = \bigwedge_{n \geq 0} F^n(x).$$

Teorema Continuity/Cocontinuity theorem

Let F be an endofunctional on a complete lattice.

1. If F is continuous,

$$\text{lfp}(F) = F^{\cup\omega}(\perp).$$

2. If F is cocontinuous,

$$\text{gfp}(F) = F^{\cap\omega}(\top).$$

Esercizio

Prove the theorem. For the cocontinuous case, first prove that $F^{\cap\omega}(\top)$ is a fixed point and then prove that it is the greatest one using the definition of meet.

If F is only monotone, in general we only know

$$\text{gfp}(F) \leq F^{\cap\omega}(\top).$$

Equality may fail.

Esempio Monotonicity without cocontinuity

Consider the order consisting of the negative integers together with $-\omega$ and $-(\omega + 1)$, ordered so

that

$$-n \geq -\omega \geq -(\omega + 1).$$

Define

$$F(-n) = -(n + 1), \quad F(-\omega) = -(\omega + 1), \quad F(-(\omega + 1)) = -(\omega + 1).$$

Then F is monotone but not cocontinuous,

$$F^{\cap \omega}(-1) = -\omega, \quad \text{gfp}(F) = -(\omega + 1).$$

With monotonicity alone, the greatest fixed point can be reached by transfinite iteration:

$$\begin{aligned} F^0(\top) &= \top, \\ F^{\lambda+1}(\top) &= F(F^\lambda(\top)), \\ F^\lambda(\top) &= \bigwedge_{\beta < \lambda} F^\beta(\top) \quad \text{for limit } \lambda. \end{aligned}$$

Iterating sufficiently far yields

$$F^\infty(\top) = \text{gfp}(F).$$

The dual result holds for least fixed points.

15.2 Continuity and cocontinuity of rule systems

A rule functional need not be continuous. For example, a rule with infinitely many premises

$$\frac{a_1 \quad a_2 \quad \cdots}{a}$$

may have its conclusion enabled in the union of a chain without being enabled in any finite element of the chain.

Definizione Finite in the premises

A set of rules R is *FP* if every rule has a finite set of premises.

Proposition

If R is FP, then Φ_R is continuous. Hence

$$\text{lfp}(\Phi_R) = \Phi_R^{\cup \omega}(\emptyset).$$

FP does not imply cocontinuity. Consider

$$X = \{b\} \cup \{a_1, a_2, \dots\}$$

and one rule a_i/b for every i . Then

$$\Phi(T) = \begin{cases} \{b\} & \exists i, a_i \in T, \\ \emptyset & \text{otherwise.} \end{cases}$$

For the decreasing chain

$$T_i = \{a_j \mid j \geq i\},$$

we have

$$\Phi\left(\bigcap_i T_i\right) = \emptyset, \quad \bigcap_i \Phi(T_i) = \{b\}.$$

Definizione Finite in the conclusions

A rule system R is *FC* if, for every conclusion x , there are only finitely many premise sets S such that $(S, x) \in R$.

Teorema

If R is FC, then Φ_R is cocontinuous. Hence

$$\text{gfp}(\Phi_R) = \Phi_R^{\cap\omega}(X).$$

Without FC we still have

$$\text{gfp}(\Phi_R) \subseteq \Phi_R^{\cap\omega}(X).$$

15.3 Iterative reading of the examples

For finite traces,

$$P \in \Phi_{R_\downarrow}^n(\emptyset)$$

if and only if there exists a sequence of at most $n - 1$ transitions from P to a stopped process. Initially the approximation is empty; then we add stopped processes, then processes having a stopped derivative, and so on. On a finite LTS the chain stabilizes after finitely many steps.

For infinite traces,

$$P \in \Phi_{R_\uparrow}^n(\text{Proc})$$

if and only if P can perform at least n consecutive transitions. We start from all processes; at the first step we remove states without derivatives, at the second step those that cannot perform two steps, and so on. On a finite system this chain also stabilizes after finitely many steps.

15.4 Approximants of bisimilarity

Define

$$\sim_0 = \text{Proc} \times \text{Proc}.$$

Inductively, $P \sim_{n+1} Q$ if every transition of P can be matched by Q with derivatives in $\sim_n$, and conversely. Put

$$\sim_\omega = \bigcap_{n \geq 0} \sim_n.$$

Thus level n compares behaviours only up to depth n .

Proposition

The sequence

$$\sim_0 \supseteq \sim_1 \supseteq \sim_2 \supseteq \dots$$

is decreasing and

$$\sim_n = F_\sim^n(\text{Proc} \times \text{Proc}), \quad \sim_\omega = F_\sim^{\cap\omega}(\text{Proc} \times \text{Proc}).$$

Approximants are also useful to prove non-bisimilarity: it is enough to find a finite depth at which the pair is removed.

Esempio $\sim_\omega$ can be larger than $\sim$

Consider states $a_0, a_1, \dots, a_\omega$ with

$$a_0 = 0, \quad a_n \xrightarrow{a} a_{n-1} \quad (n \geq 1), \quad a_\omega \xrightarrow{a} a_\omega.$$

Let P have an a -transition to every a_n , $n \geq 0$, and let Q have the same transitions plus

$$Q \xrightarrow{a} a_\omega.$$

For every finite depth n , $P \sim_n Q$, hence $P \sim_\omega Q$. However, $P \not\sim Q$: the transition of Q to a_ω can only be matched by P by choosing some finite a_n , which eventually stops and is not bisimilar to a_ω . Thus $F_\sim$ is not cocontinuous in general.

Definizione Finite branching

An LTS is *finitely branching* if every process has finitely many immediate derivatives.

Teorema

On finitely branching LTSs,

$$\sim = \sim_\omega.$$

Proof

The inclusion $\sim \subseteq \sim_\omega$ follows by showing $\sim \subseteq \sim_n$ for every n . Conversely, put

$$R = \{(P, Q) \mid P \sim_\omega Q\}.$$

Let $(P, Q) \in R$ and $P \xrightarrow{\mu} P'$. For every n , from $P \sim_{n+1} Q$ there exists

$$Q \xrightarrow{\mu} Q_n \quad \text{with} \quad P' \sim_n Q_n.$$

By finite branching, there are only finitely many possible Q_n , so one derivative Q_i occurs for infinitely many values of n . Since the approximants are decreasing,

$$P' \sim_n Q_i \quad \forall n,$$

and therefore $P' \sim_\omega Q_i$. The symmetric case is analogous, so R is a bisimulation.

15.5 Algorithms for bisimilarity

The approximant iteration underlies algorithms for checking bisimilarity and minimizing the state space of finite-state processes. The basic idea is partition refinement: initially all states are in one block, then states that cannot match one another are progressively separated.

The relevant complexity results are:

- the bisimulation problem is P-complete;
- for a system with m transitions and n states there are algorithms running in

$$O(m \log n)$$

time and $O(m + n)$ space, such as the Paige–Tarjan algorithm;

- trace equivalence and testing equivalence are PSPACE-complete.

16 Interpretation by proof trees

Definizione Proof tree

Let R be a set of ground rules over X . An X -labelled tree is a *proof tree* for x if the root is labelled x and, for every node labelled y , if S is the set of labels of its children, then

$$(S, y) \in R.$$

Proof trees may be infinite in depth or width; children of the same node are assumed to carry distinct labels. A tree is well founded if it contains no infinite path.

For lists, a finite proof builds a finite list. A periodic infinite list instead gives an infinite proof tree: it is valid coinductively but not inductively.

Teorema

$$x \in \text{lfp}(\Phi_R) \iff \text{there exists a well-founded proof tree for } x.$$

The result can be proved through approximants. If R is FP, every node has finitely many children and a well-founded proof tree has finite height; in the examples of traces, lists, and the λ -calculus, inductive objects therefore admit finite derivations. Without FP, a well-founded tree need not have finite height.

Teorema

$$x \in \text{gfp}(\Phi_R) \iff \text{there exists a proof tree, possibly non-well-founded, for } x.$$

Proof

If $x \in \text{gfp}(\Phi_R)$, then there exists a post-fixed point T with $x \in T$. For every $y \in T$, choose a rule

$$(S_y, y) \in R, \quad S_y \subseteq T.$$

Build the tree starting from x and using S_y as the set of children of every node labelled y . Conversely, given a proof tree for x , let T be the set of all node labels. For every $y \in T$, the children of the corresponding node form a set $S \subseteq T$ with $(S, y) \in R$, hence

$$y \in \Phi_R(T).$$

Therefore $T \subseteq \Phi_R(T)$, so T is post-fixed and $x \in \text{gfp}(\Phi_R)$.

17 Interpretation by games

Another reading of induction and coinduction uses a game between two players:

- the *verifier* V , who tries to construct a proof tree;
- the *refuter* R , who tries to challenge the derivation.

Given $x_0 \in X$, a play has the form

$$x_0, S_0, x_1, S_1, \dots$$

The verifier chooses S_i such that $(S_i, x_i) \in R$, that is, a rule with conclusion x_i . The refuter then chooses an element $x_{i+1} \in S_i$, challenging the verifier to justify that premise.

A play ends when one of the players cannot move, and that player loses. If the play is infinite, the convention depends on the interpretation:

1. in the inductive game the refuter wins, because proofs must be well founded;
2. in the coinductive game the verifier wins, because infinite derivations are allowed.

Esempio Game for divergence in the λ -calculus

Let

$$e_1 = \lambda x.xx.$$

For $e_1 e_1$, the verifier can produce the infinite play

$$e_1 e_1, \{e_1 e_1\}, e_1 e_1, \dots,$$

which is winning in the coinductive game.

For $e_2 = \lambda x.xxx$, an infinite play winning for V exists as well, while a bad choice may lead to a finite play won by the refuter.

Esempio Game for finite traces

For $\downarrow$, the verifier chooses at every step either a transition to a derivative or the axiom once a stopped process is reached. A cyclic choice gives an infinite play, which is losing in the inductive game; a choice leading to a stopped process eventually yields $S = \emptyset$ and is winning for the verifier.

Definizione Strategy

A strategy for the verifier maps a history ending in an element x to a rule to be used, that is, a set S with $(S, x) \in R$. A strategy for the refuter maps a history ending in a set S chosen by the verifier to an element of S to challenge. A strategy is winning if every play compatible with it is won by the player using it.

For induction and coinduction, winning strategies can be chosen history-free.

Teorema Game characterization

1. $x_0 \in \text{lfp}(\Phi_R)$ if and only if the verifier has a winning strategy in the inductive game $G^{\text{ind}}(R, x_0)$;
2. $x_0 \in \text{gfp}(\Phi_R)$ if and only if the verifier has a winning strategy in the coinductive game $G^{\text{coind}}(R, x_0)$.

17.1 The bisimulation game

Applying the general rule construction to $F_{\sim}$ produces rules whose conclusions are pairs (P, Q) . For every transition on one side, the verifier must choose a matching transition on the other side. Formally, one may choose functions f, g associating matching derivatives; the set of new pairs to be justified is

$$\text{Der}(P, Q, f, g).$$

The refuter chooses one of these pairs and the game continues. If both processes are stopped, the set of premises is empty and the verifier wins; if a required match does not exist, the refuter wins. Since this is a coinductive game, an infinite play is winning for the verifier.

An equivalent and simpler formulation lets the refuter move first. From a pair (P_i, Q_i) :

1. the refuter chooses a transition of one process;
2. the verifier must find a transition with the same label from the other process;
3. the pair of derivatives becomes the next position.

If the verifier cannot answer, the verifier loses; if the refuter has no challenge or the play continues forever, the verifier wins.

Teorema Bisimulation game

$P \sim Q \iff$ the verifier has a winning strategy from (P, Q) .

Moreover,

$P \not\sim Q \iff$ the refuter has a winning strategy from (P, Q) .

Esempio A winning strategy for the refuter

Consider a process P_3 that after an action a can enter two different branches, one offering c after b and the other offering d after b , and a process Q_3 in which the choice is postponed until after the b -transition. The refuter first selects the a -branch of P_3 , then a b -transition of Q_3 forcing an incompatible derivative, and finally exposes a d -transition that the derivative of P_3 cannot match. This is a winning strategy and proves $P_3 \not\sim Q_3$.

Part V

Concurrency models and process operators

18 Concurrent programming

A sequential program has a single thread of control. A concurrent program has multiple processes or threads working together to achieve a common goal, and can therefore perform several computations in parallel or control external activities occurring at the same time.

18.1 Communication

Concurrent threads can exchange information in two main ways.

1. *Indirect communication*: processes access shared memory. Care is required to guarantee exclusive access to shared variables.
2. *Direct communication*: processes exchange messages, possibly while running on different processors or machines.

Reasons for using concurrency include:

1. exploiting multiprocessor hardware;
2. modelling distributed systems, for example clients and servers running on different machines;
3. expressing problems that are more naturally described concurrently;
4. increasing throughput, so that an I/O call blocks only one thread rather than the whole application;
5. increasing responsiveness, for example through high-priority threads;
6. obtaining a more appropriate structure for programs that interact with the environment, control several activities, and handle several events.

Typical examples include windowing systems on PCs, embedded real-time systems in electronics, cars and telecommunications, web and database servers, and operating-system kernels.

18.2 Why concurrent programming is difficult

Concurrent programs are especially prone to errors. The Therac-25 accidents show that concurrent programming errors can have severe consequences, while problems in the interaction between concurrent tasks on the Mars Rover caused periodic software resets and reduced availability for exploration. Deadlock is another common failure mode of concurrent programs.

Esempio Concurrent search for a zero

Let $f: \mathbb{Z} \rightarrow \mathbb{Z}$ be a total function that is expensive to compute. We want to search concurrently for a zero among positive and negative integers, terminating if and only if a zero exists.

A first idea uses two threads and a shared variable `found`. If both threads independently initialize

`found := false,`

one thread may find a zero and set `found := true`, only for the late initialization of the other thread to overwrite the result. Shared variables therefore introduce interference.

A first implementation is

```
S1 = found=false; n=0;
    while(!found) { n++; found=(f(n)==0); }
```

```
S2 = found=false; z=0;
    while(!found) { z--; found=(f(z)==0); }
```

S1 || S2

If f has a positive zero but no negative zero, and S_1 finishes before S_2 starts, then S_2 resets `found` to false and searches forever for a non-existing negative zero.

Esempio Arbitrary preemption

Move the initialization before the threads are created. A problem still remains: a thread may be preempted immediately after testing the loop guard, another thread may change the shared state, and when the first thread resumes it may execute an action that is no longer consistent with the condition it observed. We cannot assume particular points at which preemption occurs.

The second implementation initializes the shared variable only once:

```
found=false; (R1 || R2)
R1 = n=0; while(!found) { n++; found=(f(n)==0); }
R2 = z=0; while(!found) { z--; found=(f(z)==0); }
```

A problematic interleaving is the following: R_2 enters the body of the loop and is preempted before decrementing z ; R_1 finds a positive zero and sets `found = true`; R_2 resumes, completes the body it had already entered, and can set `found = false` again.

Esempio Unfair scheduling

Even after removing problematic writes, a scheduler can in principle run one thread forever and never give CPU time to the other. Fairness of scheduling cannot be assumed unless it is explicitly modelled.

The third implementation removes assignments to false from the loops:

```
found=false; (T1 || T2)
T1 = n=0; while(!found) { n++; if (f(n)==0) found=true; }
T2 = z=0; while(!found) { z--; if (f(z)==0) found=true; }
```

If the scheduler indefinitely assigns the CPU to T_2 and the only zero is positive, T_1 may never run.

A fourth implementation introduces a token turn to force alternation:

```
turn=1; found=false; (Q1 || Q2)
Q1 = n=0; while(!found) {
    wait turn==1 then { turn=2; n++; if (f(n)==0) found=true; } }
Q2 = z=0; while(!found) {
    wait turn==2 then { turn=1; z--; if (f(z)==0) found=true; } }
```

If one thread finds a zero while the other is waiting for a token value that will never be produced again, the system can deadlock waiting for an impossible event.

A fifth proposal explicitly passes the token after the loop body terminates:

```
turn=1; found=false; ({P1; turn=2;} || {P2; turn=1;})
```

where P_1, P_2 are the two token-disciplined loops. Its correctness is not immediate and should be established formally. The main lessons are:

1. shared variables require care;
2. arbitrary assumptions about preemption points are invalid;
3. scheduling assumptions must be made explicit;

4. termination of one component must remain compatible with the behaviour of the others.

19 Concurrent and reactive systems

For sequential deterministic programs, a denotational semantics as a partial function

$$\text{Input} \not\rightarrow \text{Output}$$

is often adequate. Non-termination is normally undesirable and the output, when it exists, is unique.

Concurrent and reactive systems are different:

- they may have nondeterministic behaviours;
- they may have no unique observable final result;
- non-termination can be the intended behaviour, as for a server or an operating system;
- their meaning is determined by continuous interaction with the environment.

Examples include operating systems, protocols, mobile devices, vending machines, and other systems that react indefinitely to external requests.

The central questions become: how do we model the system, how do we design its composition, which properties should be verified, and how do we prove that an implementation satisfies a specification? We need mathematical models, formal languages describing possible behaviours, and verification techniques or tools, preferably automatic and compositional. Real failures such as the Intel Pentium-II floating-point division bug, the Ariane-5 crash caused by a conversion from a 64-bit real to a 16-bit integer, and the Mars Pathfinder problems motivate the use of formal methods.

19.1 Processes as an abstraction

The central abstraction is the *process*. A process exposes actions, and performing an action transforms it into another process. An LTS is therefore a natural model:

$$P \xrightarrow{a} P'.$$

A process can represent not only a program, but also a user, a communication channel, a hardware component, or the environment itself.

20 Operational models for concurrency

One possible representation uses Kripke structures, in which atomic information is associated with states. Here we focus instead on LTSs, where observable information is attached to transitions.

20.1 Automata and LTSs

A finite automaton may contain ε -transitions, namely internal moves that do not consume input symbols. For concurrent systems we similarly distinguish externally visible actions from internal actions, usually denoted by τ .

Esempio Bill and Ben

Consider

$$Q = \{q_0, q_1, q_2, q_3, q_4\}, \quad A = \{\text{play}, \text{work}, \tau\},$$

with transitions

$$\begin{array}{ll} q_0 \xrightarrow{\text{play}} q_1, & q_0 \xrightarrow{\text{work}} q_2, \\ q_1 \xrightarrow{\text{work}} q_3, & q_2 \xrightarrow{\text{play}} q_3, \\ q_3 \xrightarrow{\tau} q_4. & \end{array}$$

The actions `play` and `work` are observable, whereas τ represents an internal step.

A transition table, XML representation, or explicit graph is sufficient for small systems, but does not reveal the structure of composite systems and becomes unmanageable for large models. We therefore need syntactic operators for building processes and SOS rules for deriving the corresponding LTSs.

21 Regular expressions as a first process algebra

Regular expressions provide a simple example of a language whose semantics can be given operationally. Their syntax is

$$E ::= 0 \mid 1 \mid a \mid E + E \mid E \cdot E \mid E^*.$$

The state 1 is final, while 0 cannot perform any transition.

21.1 SOS rules

Use ε as a termination label. The rules are

$$\overline{1 \xrightarrow{\varepsilon} 1}, \quad \overline{a \xrightarrow{a} 1}.$$

For choice:

$$\frac{E \xrightarrow{\mu} E'}{E + F \xrightarrow{\mu} E'}, \quad \frac{F \xrightarrow{\mu} F'}{E + F \xrightarrow{\mu} F'}.$$

For sequentialization:

$$\frac{E \xrightarrow{\mu} E'}{E \cdot F \xrightarrow{\mu} E' \cdot F}, \quad \frac{E \xrightarrow{\varepsilon} 1}{E \cdot F \xrightarrow{\varepsilon} F}.$$

For iteration:

$$\overline{E^* \xrightarrow{\varepsilon} 1}, \quad \frac{E \xrightarrow{\mu} E'}{E^* \xrightarrow{\mu} E' \cdot E^*}.$$

The induced automaton has E as its initial state, the expressions reachable by the rules as states, and 1 as its only final state. The transition relation extends to words:

$$E \xrightarrow{s} E'$$

denotes a sequence of transitions whose visible labels concatenate to s , ignoring the appropriate ε -steps.

Esempio

For

$$a \cdot b \cdot c$$

there is a derivation performing a , then b , then c , with the required ε -steps moving from one sequential component to the next, eventually reaching 1.

The SOS recognizes the same regular language as the usual interpretation, although the generated automaton is not unique up to isomorphism. Different process descriptions of a^* , for example, may have different LTSs while recognizing the same language.

Esempio Structural derivations

For $(a + b)^*$, we derive

$$(a + b)^* \xrightarrow{a} 1 \cdot (a + b)^*$$

from $a \xrightarrow{a} 1$, first using the left choice rule and then the iteration rule. Moreover,

$$1 \cdot (a + b)^* \xrightarrow{\varepsilon} (a + b)^*$$

follows from $1 \xrightarrow{\varepsilon} 1$ and the second rule for sequentialization.

As a second example,

$$(a^* + b^*)^* \xrightarrow{b} 1 \cdot b^* \cdot (a^* + b^*)^*.$$

The derivation uses, in order, the atomic rule for b , the iteration rule for b^* , the right choice rule, and the outer iteration rule.

The semantics is the *least* LTS satisfying the rules. This is essential: an extra transition does not belong to the semantics merely because it does not syntactically contradict the rules. Only derivable transitions are included.

22 Operators for process modelling

The basic classes of operators are:

1. basic processes;
2. action prefixing;
3. sequentialization;
4. choice;
5. parallel composition and interaction;
6. abstraction and interaction delimiters;
7. recursion and other operators for infinite behaviour.

The main design questions are:

1. why structure is important;
2. which operators should be used to write processes;
3. which operators should be used to compose processes in parallel;
4. how these operators should be defined;
5. how their convenience and expressive power should be evaluated.

Operational, denotational, and algebraic semantics are all possible; SOS is especially suitable because each operator can be defined by a small number of local rules.

Esempio Three approaches to the semantics of operators

Fix a process signature. There are three main approaches.

1. In *operational semantics*, every term is interpreted as an abstract machine through its LTS; this is the approach historically emphasized by CCS.
2. In *denotational semantics*, a term is mapped by an interpretation function $\llbracket \cdot \rrbracket$ to a mathematical object in a suitable domain; this is the approach historically emphasized by CSP.
3. In *algebraic semantics*, processes are identified through equational laws; this is the approach historically emphasized by ACP.

For an operator $\text{op}(E_1, \dots, E_n)$, a typical SOS rule observes transitions of some arguments

$$E_{i_1} \xrightarrow{\alpha_1} E'_{i_1}, \dots, E_{i_m} \xrightarrow{\alpha_m} E'_{i_m}$$

and derives a transition of the compound term

$$\text{op}(E_1, \dots, E_n) \xrightarrow{\alpha} C[E'_1, \dots, E'_n],$$

where arguments not involved in the premises remain unchanged. A small collection of rules therefore defines all automata generated by the chosen operators, and the LTS of a term is always the least one satisfying those rules.

22.1 Actions and basic processes

A process can be viewed as a black box with ports through which it interacts with the environment, or as a node in a flow graph with labelled transitions. Actions can be visible or internal.

Definizione Inactive process

The process

$$0$$

or **stop** has no transitions.

A vending machine modelled by 0 is completely broken: it accepts no coins and produces nothing.

Definizione Termination

A termination process, denoted by 1 , **exit**, or $\checkmark$, performs a distinguished successful-termination action and then stops, for example

$$\text{exit} \xrightarrow{\checkmark} \text{stop}.$$

Definizione Atomic action and prefixing

An atomic action a can be represented by

$$a \xrightarrow{a} \text{stop}.$$

More generally, action prefixing is defined by

$$\frac{}{\mu.P \xrightarrow{\mu} P}.$$

For example,

$$\text{coin.choc}.0$$

accepts a coin, delivers a chocolate, and then stops.

22.2 Sequentialization

For processes with an explicit termination action $\checkmark$, sequential composition $E; F$ lets E evolve until it terminates and then continues as F . The rules are

$$\frac{E \xrightarrow{\mu} E' \quad \mu \neq \checkmark}{E; F \xrightarrow{\mu} E'; F},$$

$$\frac{E \xrightarrow{\checkmark} E'}{E; F \xrightarrow{\tau} F}.$$

The termination of the first component is therefore hidden as an internal step when control passes to the second component.

Esercizio

Consider the syntax

$$P ::= 0 \mid \surd \mid a \mid P \cdot P$$

and design SOS rules producing the intended LTS for sequentialization.

22.3 Nondeterministic choice

Definizione Choice

The choice $P + Q$ inherits transitions from both components:

$$\frac{P \xrightarrow{\mu} P'}{P + Q \xrightarrow{\mu} P'}, \quad \frac{Q \xrightarrow{\mu} Q'}{P + Q \xrightarrow{\mu} Q'}.$$

The choice is resolved by the first action performed. For example,

$$\text{coin}.\text{(choc.0 + water.0)}$$

offers the choice between chocolate and water after receiving the coin, whereas

$$\text{coin.choc.0 + coin.water.0}$$

has two coin-transitions already at the initial state, so the branch can be resolved before the environment can distinguish the alternatives.

22.3.1 Internal choice

Internal choice $P \oplus Q$ is defined by

$$P \oplus Q \xrightarrow{\tau} P, \quad P \oplus Q \xrightarrow{\tau} Q.$$

The process itself, rather than the environment, chooses the branch.

22.3.2 External choice

External choice, denoted by $\square$, is resolved by a visible action offered by one of the branches. The rules are

$$\frac{P \xrightarrow{\alpha} P'}{P \square Q \xrightarrow{\alpha} P'} \quad (\alpha \neq \tau), \quad \frac{Q \xrightarrow{\alpha} Q'}{P \square Q \xrightarrow{\alpha} Q'} \quad (\alpha \neq \tau),$$

while internal steps do not resolve the choice:

$$\frac{P \xrightarrow{\tau} P'}{P \square Q \xrightarrow{\tau} P \square Q'}, \quad \frac{Q \xrightarrow{\tau} Q'}{P \square Q \xrightarrow{\tau} P \square Q'}.$$

Thus a τ -step can evolve one branch while leaving the alternative offered by the other branch available to the environment.

Esempio External and internal choice

Consider

$$\text{coin}.\text{(choc.0} \oplus \text{water.0)} \square \text{water.0}.$$

After `coin` and an internal choice towards `choc`, we obtain

$$\text{choc}.0 \sqcap \text{water}.0,$$

so the user still has the possibility of requesting `water`. If the outer $\sqcap$ is replaced by $\oplus$, another internal step can reduce the process to `choc.0` alone, and the environment no longer retains the alternative.

Esercizio

Express $P \oplus Q$ using τ -prefixing and $+$. Draw the LTSs of different parenthesizations of a term combining $+$, $\oplus$, and external choice, for example

$$a + b \oplus c \sqcap d.$$

22.4 Parallelism without communication

Pure interleaving parallel composition, written here as $P \parallel Q$, is defined by

$$\frac{P \xrightarrow{\mu} P'}{P \parallel Q \xrightarrow{\mu} P' \parallel Q}, \quad \frac{Q \xrightarrow{\mu} Q'}{P \parallel Q \xrightarrow{\mu} P \parallel Q'}.$$

Thus the actions of the two components are arbitrarily interleaved. A process

$$m \parallel a \parallel r \parallel e$$

can generate all interleavings compatible with performing each letter once.

22.5 Milner parallelism and handshake communication

For every visible action a , introduce a co-action $\bar{a}$. Milner parallel composition $P \mid Q$ allows both interleaving and synchronization of complementary actions:

$$\frac{P \xrightarrow{\mu} P'}{P \mid Q \xrightarrow{\mu} P' \mid Q}, \quad \frac{Q \xrightarrow{\mu} Q'}{P \mid Q \xrightarrow{\mu} P \mid Q'},$$

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}.$$

Synchronization is a handshake and produces an internal action τ .

Esempio Successful and unsuccessful interaction

Consider a machine and a user, schematically

$$\text{coin} . (\text{choc}.0 \oplus \text{water}.0) \mid \overline{\text{coin}} . \overline{\text{choc}} . 0.$$

One execution synchronizes on the `coin`, lets the machine choose `choc`, and then synchronizes on chocolate, reaching $0 \mid 0$. Another execution lets the machine choose `water` internally after the `coin`. The user is waiting for `choc`, so the system becomes stuck. Parallel composition describes incompatible interactions as well as the intended ones.

Esercizio

Draw the complete LTS of the composition above, including both the successful branch and the branch leading to deadlock.

22.6 Merge with a synchronization function

A more general form of parallel composition uses a function

$$\gamma(a, b)$$

that determines the label produced by synchronizing a and b . Besides the two interleaving rules, use

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{b} Q'}{P \parallel Q \xrightarrow{\gamma(a,b)} P' \parallel Q'}$$

whenever $\gamma(a, b)$ is defined. For example, `getCoin` and `putCoin` can synchronize and produce `ok`.

Two related operators are:

- the *communication merge*, which allows only an initial synchronization;
- the *left merge*, which forces the first transition to be performed by the left component and then continues with ordinary parallel composition.

These operators make it possible to decompose parallel composition algebraically.

Esercizio

Compare

$$P \parallel Q$$

with a decomposition of the form

$$(P \triangleleft Q) + (Q \triangleleft P) + (P \mid_c Q),$$

where $\triangleleft$ denotes left merge and $\mid_c$ communication merge.

22.7 Hoare parallelism

In $P \mid [L] \mid Q$, actions in L must be performed simultaneously by both processes with the same label, while actions outside L interleave freely. In particular,

$$P \mid [\emptyset] \mid Q$$

coincides with pure interleaving.

22.8 Synchronization algebra

A *synchronization algebra* uses a structure

$$\langle \Lambda, *, 0, \bullet \rangle,$$

where $*$ is an idle action and 0 denotes forbidden synchronization. The operation $\bullet$ is associative and commutative and satisfies conditions such as

$$a \bullet 0 = 0, \quad * \bullet * = *,$$

together with the requirement that synchronization can yield $*$ only in the idle-idle case. Parallel transitions are labelled by

$$\alpha \bullet \beta$$

whenever the result is different from 0 . This formalism unifies interleaving and synchronization.

Assume also an idle transition

$$E \xrightarrow{*} E$$

for every process E . Then an ordinary transition of one component is a synchronization with the idle action of the other component. The condition

$$a \bullet b = * \implies a = b = *$$

prevents a genuine interaction from being mistaken for inactivity.

22.9 Restriction and hiding

Definizione Restriction

The operator $P \setminus L$ prevents actions in L , and their co-actions, from occurring independently towards the environment. A transition

$$P \xrightarrow{\alpha} P'$$

is inherited by $P \setminus L$ only if α is neither a restricted name nor its co-action.

Restriction can be used to *force* interaction: two components with complementary actions on a restricted channel cannot perform those actions independently towards the environment and must synchronize.

Definizione Hiding

Hiding P/L leaves actions outside L unchanged and turns actions in L into τ .

Unlike restriction, hiding does not block an action: it makes it invisible. The distinction is important because hiding should internalize behaviour rather than remove behaviour that the component can perform.

Esempio Forcing and hiding an interaction

Compose a process that emits `ok` with a machine that receives it and restrict the channel `ok`. The two complementary actions can no longer escape independently and are forced to synchronize as a τ -step. An external user trying to perform `ok` directly is blocked by restriction.

With Hoare parallelism, a signal `ok` can instead be synchronized and then hidden through

$$(P \mid [\{\text{ok}\}] \mid Q) / \{\text{ok}\}.$$

The signal is internalized: the step remains as τ , but it is no longer visible externally.

22.10 Relabelling

Given a relabelling function f ,

$$\frac{P \xrightarrow{\mu} P'}{P[f] \xrightarrow{f(\mu)} P'[f]}.$$

Relabelling lets us reuse the same component with different port names. For example, the process

`soldo.acqua.0`

can be adapted to an English interface by mapping `soldo` to `coin` and `acqua` to `water`.

23 Infinite behaviours

23.1 Recursion

Recursion can be introduced directly through $\text{rec } X.E$. Its SOS rule unfolds the recursive definition:

$$\frac{E\{\text{rec } X.E/X\} \xrightarrow{\mu} E'}{\text{rec } X.E \xrightarrow{\mu} E'}.$$

Examples include a cell alternating reads and writes, or a vending machine returning to its initial state after every sale.

Alternatively, process constants can be introduced by a set of declarations Γ :

$$X \triangleq E \in \Gamma, \quad \frac{E \xrightarrow{\mu} E'}{X \xrightarrow{\mu} E'}.$$

The two formulations can be translated into one another.

Esercizio

Using recursive constants, construct a process with states alternating **up** and **down** actions, and a process with read and write transitions on two values A, B .

23.2 Replication

Replication $!P$ represents an unbounded number of copies of P . It can be described by

$$\frac{P \mid !P \xrightarrow{\mu} P'}{!P \xrightarrow{\mu} P'}.$$

Conceptually,

$$!P \triangleq P \mid !P \triangleq \text{rec } X.(P \mid X).$$

A replicated vending machine can therefore provide chocolate an arbitrary number of times.

23.3 Iteration

The operator P^* repeats P an arbitrary number of times and also allows immediate termination. It is the process counterpart of the Kleene star for regular expressions.

Esercizio

Study the LTSs of several recursive terms $\text{rec } X.E$ and compare recursion, replication, and iteration. Analyze also how restriction changes infinite behaviours.

Esercizio Recursion, choice and parallelism

Draw the LTSs of

$$\text{rec } X.(a.X + b.X + c.0),$$

$$\text{rec } X.(a.X \oplus b.X \oplus c.0),$$

and

$$\text{rec } X.((a.X + b.X) \mid c.0).$$

Suppose moreover that P can perform a but not $\bar{a}$. Give an intuitive interpretation of

$$(P \mid \bar{a}.Q) \setminus \{a\} \quad \text{and} \quad (P \mid !\bar{a}.Q) \setminus \{a\}.$$

23.4 Linking components

Consider

$$C \triangleq \text{in.out}.C.$$

Relabel the output of one copy to a private channel p , and the input of the other copy to the same channel:

$$C_i = C[p/\text{out}], \quad C_o = C[p/\text{in}].$$

Their link is

$$C \smile C = (C_i \mid C_o) \setminus \{p\}.$$

Restriction forces communication on the internal link and hides the connection from the environment.

Esercizio

Given recursive components Z, I, D , define a composition operation using relabelling and restriction and draw its LTS.

One concrete operator is

$$X_Y \triangleq (X[p, q, r/i, d, z] \mid Y[p, q, r/\text{inc}, \text{dec}, \text{zero}]) \setminus \{p, q, r\},$$

with declarations

$$\begin{aligned} Z &\triangleq \text{zero}.Z + \text{inc}.(I_Z), \\ I &\triangleq \text{dec}.D + \text{inc}.i.I, \quad D \triangleq d.I + z.Z. \end{aligned}$$

Construct a portion of the LTS generated from Z by following the links created by relabelling and restriction.

24 Operators with data

24.1 Value passing

The input prefix

$$a(x).P$$

receives a value v and continues as $P\{v/x\}$:

$$a(x).P \xrightarrow{a(v)} P\{v/x\}.$$

The output prefix

$$\bar{a}e.P$$

evaluates the expression and sends its value. Input and output actions carrying the same value can synchronize and produce τ .

Formally,

$$a(x).E \xrightarrow{a(v)} E\{v/x\}, \quad \bar{a}e.E \xrightarrow{\bar{a} \text{ val}(e)} E,$$

and the two symmetric interaction rules are

$$\frac{E \xrightarrow{\bar{a}v} E' \quad F \xrightarrow{a(v)} F'}{E \mid F \xrightarrow{\tau} E' \mid F'}, \quad \frac{E \xrightarrow{a(v)} E' \quad F \xrightarrow{\bar{a}v} F'}{E \mid F \xrightarrow{\tau} E' \mid F'}.$$

24.2 Conditional

A construct

$$\text{if } e \text{ then } P \text{ else } Q$$

chooses its branch according to $\text{val}(e)$.

Esempio Parameterized vending machine

Consider

$$\text{coin}(x).\text{if } x \geq 20 \text{ then choc.nil else nil.}$$

When interacting with a customer that sends 40, input and output on `coin` synchronize as a τ -step; the conditional then selects the branch offering `choc`.

The customer can be written as $\overline{\text{coin}}\ 40.\overline{\text{choc}}.0$. After transmitting the value, the test $40 \geq 20$ is true and the subsequent interaction on chocolate reaches $0 \mid 0$.

25 Advanced control operators

25.1 Pipelining

The operator $E > F$ executes E ; whenever E terminates, a fresh copy of F is spawned in parallel while E continues to evolve. The rules are

$$\frac{E \xrightarrow{\mu} E'}{E > F \xrightarrow{\mu} E' > F} \quad (\mu \neq \checkmark),$$

and

$$\frac{E \xrightarrow{\checkmark} E'}{E > F \xrightarrow{\tau} (E' > F) \mid F}.$$

For example,

$$(\text{coin} \mid \text{coin} \mid \text{coin}) > \text{choc}$$

spawns a copy of `choc` whenever a component on the left terminates successfully.

25.2 Disabling

The operator $E[> F$ allows E to evolve normally, but lets F interrupt it. The rules are

$$\frac{E \xrightarrow{\mu} E'}{E[> F \xrightarrow{\mu} E'[> F} \quad (\mu \neq \checkmark),$$

$$\frac{E \xrightarrow{\checkmark} E'}{E[> F \xrightarrow{\checkmark} E'}, \quad \frac{F \xrightarrow{\mu} F'}{E[> F \xrightarrow{\mu} F'}.$$

Thus an action of the second process permanently discards the residual behaviour of the first. For example,

$$(\text{coin.choc}.0)[> (\text{bang.choc}.0)$$

models a machine that can be interrupted by a customer hitting it and obtaining chocolate without inserting a coin.

25.3 Try/catch

The operator $\text{try } E \text{ catch}(A) F$ runs E until an exceptional action in A occurs. Its rules are

$$\frac{E \xrightarrow{\mu} E'}{\text{try } E \text{ catch}(A) F \xrightarrow{\mu} \text{try } E' \text{ catch}(A) F} \quad (\mu \notin A),$$

$$\frac{E \xrightarrow{\mu} E'}{\text{try } E \text{ catch}(A) F \xrightarrow{\tau} F} \quad (\mu \in A),$$

and, for normal termination,

$$\frac{E \xrightarrow{\check{}} E'}{\text{try } E \text{ catch}(A) F \xrightarrow{\check{}} E'}.$$

This is not a conventional programming-language exception mechanism: the exceptional event is observed as a process action and then converted into an internal transfer to the handler.

Esercizio Buffer, producer and consumer

Consider the following sequence of exercises.

1. Model a one-place buffer with input ports **put**, **get** and output ports **empty**, **full**, **error**. It must emit **error** if **get** is attempted on an empty buffer or **put** on a full buffer.
2. Model a producer with input ports **empty**, **full** and output port **put**, checking that the buffer is empty before performing **put**.
3. Model a consumer, deciding which ports are required and which check should precede **get**.
4. Compose buffer, producer, and consumer in parallel, restricting every port except **error**. Generate the LTS and verify that **error** is never emitted.
5. Add a second consumer and determine whether the system can now emit **error**.

Esercizio A non-trivial LTS

Define a process whose LTS reproduces, up to some internal actions, a recursive structure of states with **read** actions and values A, B . Use the process operators to hide implementation details while retaining the required visible reading behaviour.

Part VI

Behavioural equivalences

26 Specifications, implementations and equivalences

Assume we are given an abstract specification $Spec$, and we build an implementation Imp by assembling interacting components. The natural question is whether

$$Imp \quad \text{and} \quad Spec$$

have the same behaviour. To answer this question we must choose a notion of behavioural equivalence, prove the two processes equivalent, or find a counterexample and modify either the implementation or the chosen equivalence.

A reasonable equivalence should abstract from concrete state names and identify at least isomorphic LTSs, reflect communication potential, be an equivalence relation, and ideally be a congruence for the operators of the language. Depending on the application, we may also require sensitivity to deadlock, branching structure, internal actions, and other properties.

Exact equality or isomorphism of the associated LTSs is usually too fine. For example,

$$X = a.X, \quad Y = a.a.Y$$

have different LTSs, but both can only execute an infinite sequence of a -actions and should therefore be identified by a suitable behavioural equivalence.

More precisely, a useful behavioural equivalence should:

1. abstract from state identities and observe actions;
2. allow abstraction from internal behaviour when τ is unobservable;
3. identify at least isomorphic LTSs;
4. distinguish processes having different sets of action sequences;
5. allow an equivalent subprocess to replace another without changing the overall semantics;
6. be sensitive to deadlock: if one process can deadlock after a trace s , the other should be able to deadlock after the same trace, and conversely.

It should moreover be reflexive, symmetric, and transitive.

Definizione Congruence

An equivalence $\equiv$ is a congruence for a process language if for every context $C[-]$,

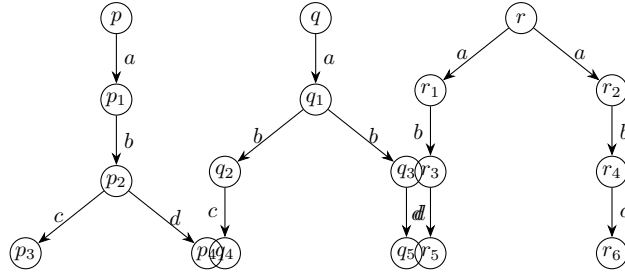
$$P \equiv Q \implies C[P] \equiv C[Q].$$

Esempio Three processes with the same traces

Consider

$$\begin{aligned} p &= a.b.(c.0 + d.0), \\ q &= a.(b.c.0 + b.d.0), \\ r &= a.b.c.0 + a.b.d.0. \end{aligned}$$

The three terms can generate the same complete action sequences, but the nondeterministic choice is resolved at different points of the computation. Hence an equivalence based only on traces loses information about branching.



27 Trace equivalence

Definizione Traces of a process

The set of traces of a process q is

$$T(q) = \{a_1 \cdots a_n \mid \exists q_1, \dots, q_n, q \xrightarrow{a_1} q_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} q_n\}.$$

The empty trace is included.

Definizione Trace equivalence

Two processes are trace equivalent if

$$p \simeq_T q \iff T(p) = T(q).$$

Two vending machines may have exactly the same observable action sequences while making internal decisions at different points. For properties depending on what remains possible after an action, trace equivalence is therefore too coarse.

28 Strong bisimulation

Definizione Strong bisimulation

A relation $R \subseteq \text{Proc} \times \text{Proc}$ is a *strong bisimulation* if, for every $(p, q) \in R$ and every action a :

1. if $p \xrightarrow{a} p'$, then there exists q' such that

$$q \xrightarrow{a} q' \quad \text{and} \quad (p', q') \in R;$$

2. if $q \xrightarrow{a} q'$, then there exists p' such that

$$p \xrightarrow{a} p' \quad \text{and} \quad (p', q') \in R.$$

Definizione Strong bisimilarity

Two processes are strongly bisimilar, written

$$p \sim q,$$

if there exists a strong bisimulation R containing (p, q) . Equivalently, $\sim$ is the union of all strong bisimulations.

Esempio Why the three trace-equivalent processes are not all bisimilar

In

$$a.b.(c.0 + d.0)$$

after a and b the residual state offers both c and d . In a process where the choice between the two branches has already been resolved, the corresponding derivative offers only one of the two actions. A bisimulation relation cannot preserve the matching against all choices of an attacker.

Esempio A non-functional bisimulation relation

A bisimulation between two LTSs can contain, for example,

$$R = \{(p_0, q_0), (p_0, q_2), (p_1, q_1), (p_2, q_1)\}.$$

A single state can therefore be related to several states of the other system: a bisimulation is not a function, but a relation grouping states with the same behaviour.

Teorema Properties of bisimilarity

Strong bisimilarity

1. is an equivalence relation;
2. is the greatest strong bisimulation;
3. satisfies the local characterization

$$p \sim q$$

if and only if every transition of either process can be matched by the other with bisimilar derivatives.

Esempio Proving non-bisimilarity

Suppose p_0 can perform an action to a state p_1 offering a c -transition, while every possible matching move of q_0 leads to a state q_1 that cannot perform c . Then no bisimulation can contain (p_0, q_0) .

28.1 Operations on bisimulations

Consider the relations

$$\emptyset, \quad \text{Id}, \quad \text{Proc} \times \text{Proc}, \quad R^{-1}, \quad R \circ S, \quad R \cup S, \quad R \cap S.$$

An arbitrary union of bisimulations is again a bisimulation, which justifies defining $\sim$ as their union. Intersections do not enjoy the same property in general.

Esercizio Bisimilarity from a diagram

Given an LTS with two initial states P, Q and several action branches, determine whether they are bisimilar. Either construct a relation containing (P, Q) , or find a transition forcing a mismatch and use it as a counterexample.

29 Bisimulation game

The bisimulation game gives a conceptual procedure for proving both bisimilarity and non-bisimilarity. There are two players:

- the attacker chooses one of the two processes and one of its transitions;
- the defender must reply with a transition carrying the same label from the other process.

The game continues from the pair of derivatives. If the defender cannot reply, the defender loses; if the defender can reply forever, or the attacker has no move, the defender wins.

Teorema

$p \sim q \iff$ the defender has a winning strategy in the bisimulation game from (p, q) .

To prove non-bisimilarity it is often easier to describe a winning strategy for the attacker than to enumerate all possible relations, of which there are

$$2^{|\text{Proc}|^2}$$

in the finite case.

30 Simulation and similarity

Definizione Simulation

A relation R is a simulation if

$$pRq \wedge p \xrightarrow{a} p' \implies \exists q', q \xrightarrow{a} q' \wedge p'Rq'.$$

No symmetric condition is required.

Write

$$p \sqsubseteq q$$

if there exists a simulation containing (p, q) . This relation is a preorder and is called *similarity*.

Double similarity is defined by

$$p \simeq q \iff p \sqsubseteq q \wedge q \sqsubseteq p.$$

It is coarser than bisimilarity: if $p \sim q$, then the two processes simulate one another, but the converse can fail because the two simulations may use different relations.

Esempio Double similarity without bisimilarity

Consider processes of the form

$$a.b.0 + a.(b.0 + c.0) \quad \text{and} \quad a.(b.0 + c.0).$$

Each process can simulate the other by choosing a suitable branch, but there is no single symmetric relation satisfying all the matching obligations required by bisimulation.

Esempio Simulation and double similarity

Similarity $\sqsubseteq$ is in general a preorder and need not be symmetric. Double similarity can equivalently be written as

$$\simeq \triangleq \sqsubseteq \cap \sqsubseteq^{-1}.$$

It is symmetric by construction and should not be confused with simulation itself.

31 Testing equivalences

An alternative approach describes behaviour through *observers* or tests. An observer is a process that interacts with the process under test and has a distinguished success action ω . Its action set is therefore

$$A_\omega = A \cup \{\omega\}.$$

31.1 Observations and success

An observation of q by o is a maximal computation of the synchronized system (q, o) . Denote the set of observations by

$$\text{OBS}(q, o).$$

An observation is *successful* if, in one of the reached configurations, the observer can perform ω .

Definizione May and must testing

We say that

1. q *may pass* o if at least one observation is successful;
2. q *must pass* o if every maximal observation is successful.

Two processes are *may equivalent* if they pass exactly the same tests in the may sense, and *must equivalent* if they pass exactly the same tests in the must sense. Testing equivalence combines the two components.

Esempio Difference between may and must

Processes offering the same traces may be may equivalent because there is always some branch allowing the observer to succeed, but fail to be must equivalent if one process can internally choose a branch from which success becomes impossible. In a representative example, p is may equivalent to q but not must equivalent, while q and r agree with respect to must and hence with respect to the testing equivalence under consideration.

In that example,

$$p \simeq_m q, \quad p \not\simeq_M q, \quad q \simeq_M r, \quad q \simeq_{\text{test}} r.$$

32 Failures, futures and acceptance sets

Definizione Refusal and failure

A set of actions L is a *refusal* of a state if the state can refuse every action in L , namely none of them is initially available. A *failure* is a pair

$$(s, L)$$

such that after trace s the process can reach a state refusing L .

Failure equivalence compares sets of failures. In the setting considered here,

$$\text{must equivalence} \equiv \text{failure equivalence},$$

whereas

$$\text{may equivalence} \equiv \text{trace equivalence}.$$

Another description uses *futures*, namely what remains possible after a trace, and acceptance sets. For a process p ,

$$\text{Init}(p) = \{a \mid \exists p', p \xrightarrow{a} p'\}$$

is the set of initially available actions. Must behaviour can be characterized by comparing, after every trace, the families of action sets that must remain acceptable.

33 Internal actions and weak equivalences

Strong bisimulation treats τ as an ordinary label and therefore distinguishes internal details that are often intended to be unobservable. We want to abstract from sequences of internal steps, including internal grinding.

33.1 Weak transitions

For a visible action a , define

$$\xRightarrow{a} \triangleq (\xrightarrow{\tau})^* \circ \xrightarrow{a} \circ (\xrightarrow{\tau})^*.$$

For the empty trace,

$$\xRightarrow{\varepsilon} \triangleq (\xrightarrow{\tau})^*.$$

More generally, a weak trace allows any number of τ -steps before, after, and between visible actions.

Definizione Weak trace equivalence

Two processes are weak trace equivalent if they have the same sets of visible traces when τ -steps are ignored.

Testing can similarly be weakened by allowing the process and observer to perform independent internal steps between visible synchronizations.

33.2 Weak bisimulation

A general formulation matches weak transitions for arbitrary traces. The following local characterization is sufficient. Define

$$\hat{\mu} = \begin{cases} \varepsilon & \mu = \tau, \\ \mu & \mu \neq \tau. \end{cases}$$

Definizione Weak bisimulation

A symmetric relation R is a weak bisimulation if

$$pRq \wedge p \xrightarrow{\mu} p'$$

implies the existence of q' such that

$$q \xRightarrow{\hat{\mu}} q' \quad \text{and} \quad p'Rq'.$$

Weak bisimilarity is written

$$p \approx q.$$

Processes differing only by redundant internal steps can therefore be weakly bisimilar. Weak bisimulation nevertheless preserves visible branching structure and does not collapse all distinctions in the way trace equivalence does.

33.3 Branching bisimulation

Branching bisimulation preserves more precisely the point at which an observable choice is resolved.

Definizione Branching bisimulation

A symmetric relation R is a branching bisimulation if, whenever

$$pRq \quad \text{and} \quad p \xrightarrow{\mu} p',$$

one of the following holds:

1. $\mu = \tau$ and $p'Rq$;
2. there exist q'', q' such that

$$q \xRightarrow{\varepsilon} q'' \xrightarrow{\mu} q',$$

with

$$pRq'', \quad p'Rq'.$$

Intermediate states traversed through τ -steps before the matching action must therefore remain related to the source state, preserving the branching structure.

Esempio Testing and bisimulation do not coincide

There are complementary counterexamples. In one, two systems are weakly testing equivalent but are neither weakly bisimilar nor branching bisimilar. In another, two systems containing a τ -loop are weakly and branching bisimilar but not testing equivalent. Thus testing-based observational notions and branching-matching notions are incomparable in general.

33.4 The spectrum of equivalences

The equivalences form a spectrum. Strong bisimilarity is among the finest notions; branching and weak bisimilarity progressively abstract from internal steps; failure/must, trace/may, and other linear-time equivalences are coarser. For strongly convergent systems, the corresponding strong and weak notions collapse in the expected way, in particular must with failures and may with traces. The appropriate equivalence therefore depends on which aspects of behaviour are intended to remain observable.

Part VII

CCS

34 A Calculus for Communicating Systems

CCS collects in a single language the main ingredients introduced so far: LTSs as behavioural models, process-composition operators, SOS rules, and behavioural equivalences. Its sequential fragment contains the nil process, action prefixing, process names with recursive definitions, and nondeterministic choice. Using these operators, every finite LTS can be described up to isomorphism. Adding parallel composition, restriction, and relabelling yields a language for communicating systems.

34.1 Names, co-names and actions

Let $\mathcal{A}$ be a set of channel names. For every $a \in \mathcal{A}$, there is a co-name $\bar{a}$. Put

$$\mathcal{L} = \mathcal{A} \cup \bar{\mathcal{A}}, \quad \text{Act} = \mathcal{L} \cup \{\tau\}.$$

Complementation satisfies

$$\overline{\bar{a}} = a, \quad \overline{\tau} = \tau.$$

Let $\mathcal{K}$ be a set of process constants.

34.2 Syntax

The syntax of CCS is

$$P ::= K \mid \alpha.P \mid \sum_{i \in I} P_i \mid P_1 \mid P_2 \mid P \setminus L \mid P[f],$$

where

$$K \in \mathcal{K}, \quad \alpha \in \text{Act}, \quad L \subseteq \mathcal{A},$$

and f is a relabelling function preserving τ and complementation:

$$f(\tau) = \tau, \quad f(\bar{a}) = \overline{f(a)}.$$

Binary choice is

$$P + Q \triangleq \sum_{i \in \{1,2\}} P_i,$$

while

$$\text{nil} \triangleq \sum_{i \in \emptyset} P_i.$$

The usual precedence conventions bind restriction and relabelling most tightly, then prefixing, then parallel composition, and finally summation.

A process constant is equipped with a defining equation

$$K \triangleq P.$$

Definitions may be recursive, for example

$$A \triangleq a.A \mid A.$$

35 SOS of CCS

The states of the CCS LTS are process terms and its labels are the actions in Act. The main SOS rules are the following.

35.1 Prefix and summation

$$\frac{}{\alpha.P \xrightarrow{\alpha} P} \quad (\text{ACT})$$

For summation,

$$\frac{P_j \xrightarrow{\alpha} P'}{\sum_{i \in I} P_i \xrightarrow{\alpha} P'} \quad j \in I. \quad (\text{SUM})$$

35.2 Parallel composition

Independent moves are described by

$$\frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q}, \quad (\text{COM1})$$

$$\frac{Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}. \quad (\text{COM2})$$

Synchronization is

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}. \quad (\text{COM3})$$

Esempio Parallel composition of complementary prefixes

The process

$$a.\text{nil} \mid \bar{a}.\text{nil}$$

can

1. perform a independently;
2. perform $\bar{a}$ independently;
3. synchronize the two actions and perform

$$a.\text{nil} \mid \bar{a}.\text{nil} \xrightarrow{\tau} \text{nil} \mid \text{nil}.$$

35.3 Restriction

$$\frac{P \xrightarrow{\alpha} P' \quad \alpha, \bar{\alpha} \notin L}{P \setminus L \xrightarrow{\alpha} P' \setminus L}. \quad (\text{RES})$$

A restricted name and its co-name cannot be exposed independently to the environment. Internal τ -actions are not blocked.

35.4 Relabelling

$$\frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}. \quad (\text{REL})$$

35.5 Constants

If

$$K \triangleq P$$

and $P \xrightarrow{\alpha} P'$, then

$$\frac{P \xrightarrow{\alpha} P'}{K \xrightarrow{\alpha} P'}. \quad (\text{CON})$$

Esempio An SOS derivation

Let

$$A \triangleq a.A.$$

A transition of a compound term such as

$$((A \mid a.\text{nil}) \mid b.\text{nil})[c/a]$$

is derived by first obtaining an a -transition from the selected component, lifting it through the parallel compositions, and finally applying the relabelling, which changes the label into c .

36 The 50 prisoners problem

CCS can be used to model the well-known 50 prisoners problem. There are fifty prisoners. One at a time, according to a fair schedule, they are taken to a room containing a light bulb whose initial state is unknown. At some point a prisoner may declare that every prisoner has already visited the room; an incorrect declaration makes all prisoners lose.

36.1 A first model

Assume first that the light is initially on. One possible light process is

$$\text{Light} \triangleq \text{turnOff}.\text{turnOn}.\text{Light}.$$

A counter can initially be represented by

$$C_{i+1} \triangleq \text{turnOff}.C_i + \tau.C_{i+1}, \quad C_0 \triangleq \text{freeAll}.$$

A generic prisoner may have a phase in which the light is turned on once, followed by a phase in which the prisoner no longer contributes:

$$P \triangleq \text{turnOn}.IP + \tau.P, \quad IP \triangleq \tau.IP.$$

This model has a problem: prisoners may always select τ and never interact with the light. Replace the inactive branches by actions that respect the light state:

$$C_{i+1} \triangleq \text{turnOff}.C_i + \text{turnOn}.\text{turnOff}.C_{i+1}, \\ P \triangleq \text{turnOn}.IP + \text{turnOff}.\text{turnOn}.P, \quad IP \triangleq \tau.IP.$$

The complete system is obtained by putting the light, the counter, and the prisoners in parallel and restricting the channels `turnOn` and `turnOff`.

The model still exposes two issues. A prisoner may perform internal behaviour indefinitely, so fairness must be represented correctly. Moreover, the fact that only one prisoner at a time can be in the room must be modelled explicitly.

36.2 Mutual exclusion

Introduce a room process

$$\text{Room} \triangleq \text{roomIn}.\text{roomOut}.\text{Room}.$$

Expand the prisoners with synchronization actions for entering and leaving the room. Mutual exclusion is then enforced by handshake communication rather than assumed implicitly.

In particular,

$$C_{i+1} \triangleq \text{roomIn}.\text{turnOff}.\text{roomOut}.C_i + \text{turnOn}.\text{turnOff}.\text{roomOut}.C_{i+1},$$

$$P \triangleq \text{roomIn}.\text{turnOn}.\text{roomOut}.IP + \text{turnOff}.\text{turnOn}.\text{roomOut}.P,$$

$$IP \triangleq \text{roomIn}.\tau.\text{roomOut}.IP.$$

The complete system restricts

$$\{\text{turnOn}, \text{turnOff}, \text{roomIn}, \text{roomOut}\}.$$

36.3 Unknown initial state

If the initial state of the bulb is unknown, model the light by

$$\text{Light} \triangleq \tau.\text{LightOn} + \tau.\text{LightOff},$$

$$\text{LightOn} \triangleq \text{turnOff}.\text{LightOff}, \quad \text{LightOff} \triangleq \text{turnOn}.\text{LightOn}.$$

The counter cannot know whether the first switch-off corresponds to an actual contribution from another prisoner. The ambiguity can be removed by making every non-counter prisoner contribute twice and having the counter count 98 switch-offs. Indeed,

$$50 + 49 = 99, \quad 49 + 49 = 98,$$

whereas if at least one prisoner had never entered, the maximum would be

$$49 + 48 = 97$$

with the light initially on, or $48 + 48 = 96$ at the other extreme. Thus define

$$P \triangleq \text{roomIn}.\text{turnOn}.\text{roomOut}.RP + \text{turnOff}.\text{turnOn}.\text{roomOut}.P,$$

$$RP \triangleq \text{roomIn}.\text{turnOn}.\text{roomOut}.IP + \text{turnOff}.\text{turnOn}.\text{roomOut}.RP,$$

$$IP \triangleq \text{roomIn}.\tau.\text{roomOut}.IP,$$

and use C_{98} in the full system.

37 Value-passing CCS

In value-passing CCS, a handshake transfers data as well as control. For instance, a bank process may receive a request carrying an identifier or an amount, process the value, and reply on the complementary channel.

Value-passing CCS can be encoded in standard CCS by replacing a parameterized input with a sum, possibly infinite, over all values:

$$a(x).P \rightsquigarrow \sum_{v \in V} a(v).P\{v/x\}.$$

This translation may produce an infinite LTS from a compact syntactic description. CCS is expressive enough to be Turing complete.

Esempio Bank

Consider

$$\overline{\text{pay}}(6).0 \mid \text{pay}(x).\overline{\text{save}}(x/2).0 \mid \text{Bank}(100).$$

After synchronization on **pay**, the second process becomes $\overline{\text{save}}(3).0$; a second synchronization leads to $\text{Bank}(103)$, where

$$\text{Bank}(\text{total}) \triangleq \text{save}(x).\text{Bank}(\text{total} + x).$$

38 Buffers and bisimilarity

38.1 Finite-capacity buffers

A buffer of capacity 1 can be defined by

$$B_0^1 \triangleq \text{in}.B_1^1, \quad B_1^1 \triangleq \text{out}.B_0^1.$$

For capacity n :

$$\begin{aligned} B_0^n &\triangleq \text{in}.B_1^n, \\ B_n^n &\triangleq \text{out}.B_{n-1}^n, \\ B_i^n &\triangleq \text{in}.B_{i+1}^n + \text{out}.B_{i-1}^n, \quad 0 < i < n. \end{aligned}$$

Teorema

A buffer of capacity n is strongly bisimilar to n one-place buffers in parallel:

$$B_0^n \sim \underbrace{B_0^1 \mid \cdots \mid B_0^1}_{n \text{ copies}}.$$

Proof Relation based on total occupancy

In the parallel composition of n cells, every component is either empty or full. Relate B_i^n with all parallel configurations containing exactly i full cells. An in-action increases occupancy by one in both systems, and an out-action decreases it by one. The resulting relation is a bisimulation.

Esercizio

Prove the usual equivalence properties of bisimilarity and show

$$B_0^{n+k} \sim B_0^n \mid B_0^k.$$

38.2 A digression on “advanced proof methods”

The following humorous list illustrates methods that are not substitutes for a proof:

1. proof by obviousness: “so evident it needs not to be mentioned”;
2. proof by general agreement: “all in favor?”;
3. proof by majority, when general agreement is unavailable;
4. proof by plausibility: “it sounds good”;
5. proof by intuition: “I have this feeling...”;
6. proof by lost reference: “I saw it somewhere”;
7. proof by obscure reference: an untraceable citation in old proceedings;
8. proof by logic: “it is in the textbook, hence it must be true”;
9. proof by intimidation: “who is saying that it is false!?”;
10. proof by authority: “Don Knuth said it was true”;
11. proof by deception: “everybody please turn their backs...”;
12. proof by divine word: “Lord said let it be true”.

For bisimulation, the actual relation must be exhibited and its matching conditions checked; intuition from a picture does not replace the proof.

39 Congruence of strong bisimilarity

Teorema Congruence theorem

Strong bisimilarity is preserved by CCS operators. If $P \sim Q$, then for every appropriate context,

$$\begin{aligned} \alpha.P &\sim \alpha.Q, \\ P + R &\sim Q + R, \\ P \mid R &\sim Q \mid R, \\ P \setminus L &\sim Q \setminus L, \\ P[f] &\sim Q[f]. \end{aligned}$$

The standard proof technique extends the original bisimulation with pairs obtained by placing related processes in the same context, and then checks the SOS rules.

For choice, this yields algebraic laws such as

$$\begin{aligned} P + Q &\sim Q + P, \\ (P + Q) + R &\sim P + (Q + R), \\ P + \text{nil} &\sim P, \\ P + P &\sim P. \end{aligned}$$

For parallel composition,

$$P \mid 0 \sim P, \quad P \mid Q \sim Q \mid P, \quad (P \mid Q) \mid R \sim P \mid (Q \mid R).$$

Proof Commutativity of choice

Consider

$$R = \text{Id} \cup \{(P + Q, Q + P) \mid P, Q \in \text{Proc}\}.$$

Every transition of $P + Q$ originates from either P or Q ; the same transition is available in $Q + P$ through the symmetric summation rule and has the same derivative. The converse direction is identical, so R is a bisimulation.

40 Cells and FIFO buffers

We compare different ways of building memory and buffers.

40.1 Cells

A one-place cell has two conceptual states, empty and full:

$$\text{Empty} \xrightarrow{\text{in}} \text{Full}, \quad \text{Full} \xrightarrow{\text{out}} \text{Empty}.$$

A capacity-two cell has three occupancy levels. Compare:

1. a directly defined cell of capacity two;
2. $\text{ParCell} = \text{Cell}_1 \mid \text{Cell}_1$;
3. SeqCell , obtained by linking two cells in series through a private channel p .

Determine which constructions are equivalent under the behavioural equivalences introduced earlier.

40.2 FIFO with binary data

For values 0, 1, a capacity-one FIFO distinguishes the states $[]$, $[0]$, and $[1]$. A capacity-two FIFO has states

$$[], [0], [1], [00], [10], [01], [11]$$

with actions in0 , in1 , out0 , and out1 respecting FIFO order. Compare the direct FIFO with

$$\text{ParFIFO} = \text{FIFO}_1 \mid \text{FIFO}_1$$

and with a serial composition using private channels p_0, p_1 . Two independent cells in parallel do not automatically preserve data order, whereas a serial connection can.

41 Weak bisimilarity in CCS

Recall

$$\xrightarrow{a} = (\xrightarrow{\tau})^* \circ \xrightarrow{a} \circ (\xrightarrow{\tau})^*, \quad a \neq \tau,$$

and

$$\xrightarrow{\varepsilon} = (\xrightarrow{\tau})^*.$$

Moreover, put

$$\hat{a} = a, \quad \hat{\tau} = \varepsilon.$$

Weak bisimulation uses $\xRightarrow{\hat{\mu}}$ to match a transition labelled μ . Typical equivalences include

$$a.\tau.P \approx a.P,$$

as well as other laws eliminating redundant internal steps or unobservable internal loops.

Weak bisimilarity is an equivalence relation, but it is not a congruence for choice $+$. It is preserved by the other CCS operators under the usual conditions. This failure for choice motivates the introduction of observational congruence in the axiomatization of weak equivalences.

42 A communication protocol

Consider the abstract specification

$$\text{Spec} \triangleq \text{acc.del.Spec.}$$

The implementation is

$$\text{Impl} = (\text{Send} \mid \text{Med} \mid \text{Rec}) \setminus \{\text{send, trans, ack, error}\}.$$

The components are

$$\begin{aligned} \text{Send} &\triangleq \text{acc.Sending}, \\ \text{Sending} &\triangleq \text{send.Wait}, \\ \text{Wait} &\triangleq \text{ack.Send} + \text{error.Sending}, \\ \text{Rec} &\triangleq \text{trans.Del}, \\ \text{Del} &\triangleq \text{del.Ack}, \\ \text{Ack} &\triangleq \text{ack.Rec}, \\ \text{Med} &\triangleq \text{send.Med}_0, \\ \text{Med}_0 &\triangleq \tau.\text{Err} + \text{trans.Med}, \\ \text{Err} &\triangleq \text{error.Med}. \end{aligned}$$

The medium can either deliver the message correctly or report an error, in which case the sender retries.

Esercizio Correctness of the protocol

Draw the LTSs of *Spec* and *Impl*, and prove

$$\text{Impl} \approx \text{Spec}.$$

The proof must abstract from the internal send, transmission, acknowledgement, and error steps, leaving externally only the cycle **acc.del**.

Part VIII

Equational axiomatizations

43 Why axiomatize an equivalence

Checking directly whether two process terms are equivalent can be expensive: their LTSs must be constructed and then an appropriate behavioural-equivalence relation must be established. An equational axiomatization allows us instead to reason syntactically on terms, often avoiding the explicit computation of LTSs and bisimulation relations. This is useful for automated reasoning and also gives a deeper understanding of the semantic effect of an equivalence.

We seek an equational theory E_{CCS} such that

$$E_{CCS} \vdash P = Q$$

is sound and complete for strong bisimilarity.

Definizione Soundness and completeness

An axiomatization E is

1. *sound* for $\sim$ if

$$E \vdash P = Q \implies P \sim Q;$$

2. *complete* for $\sim$ if

$$P \sim Q \implies E \vdash P = Q.$$

Soundness ensures that the axioms never identify terms belonging to different bisimulation classes. Completeness ensures that every pair of bisimilar terms can be equated by equational reasoning. We will seek analogous results for weak equivalences.

44 Equational logic

44.1 Signatures and terms

A signature Σ contains function symbols with fixed arities; symbols of arity zero are constants. Write

$$T(\Sigma)$$

for the set of terms, possibly containing variables, and

$$CT(\Sigma)$$

for the set of closed terms.

A substitution maps variables to terms and extends homomorphically to all terms. A *closed substitution* maps every variable to a closed term. We write

$$t\{u/x\}$$

for substituting u for x , and similarly for simultaneous substitutions.

Definizione Axiom system

An equational axiomatization E is a set of equations

$$s = t$$

between terms over the signature. It is finite if it contains finitely many axioms.

The derivable equality $=_E$ is the least relation containing every substitution instance of the axioms and closed under:

1. reflexivity;
2. symmetry;
3. transitivity;
4. substitution;
5. contexts, namely replacing equals by equals inside the same operator.

Equational logic can therefore be presented by the usual inference rules

$$\frac{}{t = t}, \quad \frac{t = u}{u = t}, \quad \frac{t = u \quad u = v}{t = v},$$

together with congruence rules for the function symbols and substitution instances of the axioms.

45 Axioms for Basic CCS

For the fragment with nil, prefixing, and summation, the basic axioms are

$$\begin{aligned} A1 \quad & P + Q = Q + P, \\ A2 \quad & (P + Q) + R = P + (Q + R), \\ A3 \quad & P + \text{nil} = P, \\ A4 \quad & P + P = P. \end{aligned}$$

Thus $+$ behaves like idempotent union of initial alternatives.

Esempio Equational calculation

Consider

$$\begin{aligned} a.(b.\text{nil} + \text{nil}) + (a.\text{nil} + a.b.\text{nil}) &= a.b.\text{nil} + (a.\text{nil} + a.b.\text{nil}) & (A3) \\ &= a.b.\text{nil} + (a.b.\text{nil} + a.\text{nil}) & (A1) \\ &= (a.b.\text{nil} + a.b.\text{nil}) + a.\text{nil} & (A2) \\ &= a.b.\text{nil} + a.\text{nil} & (A4). \end{aligned}$$

Hence

$$\text{BaseAX} \vdash a.(b.\text{nil} + \text{nil}) + (a.\text{nil} + a.b.\text{nil}) = a.b.\text{nil} + a.\text{nil}.$$

46 Restriction and relabelling

Restriction and relabelling can be pushed through summation and underneath prefixes by equational laws. For restriction:

$$\begin{aligned} \text{nil} \setminus L &= \text{nil}, \\ (P + Q) \setminus L &= P \setminus L + Q \setminus L, \\ (\alpha.P) \setminus L &= \begin{cases} \alpha.(P \setminus L) & \alpha, \bar{\alpha} \notin L, \\ \text{nil} & \text{if the action is blocked by } L. \end{cases} \end{aligned}$$

For relabelling:

$$\begin{aligned}\text{nil}[f] &= \text{nil}, \\ (P + Q)[f] &= P[f] + Q[f], \\ (\alpha.P)[f] &= f(\alpha).P[f].\end{aligned}$$

These laws eliminate structural operators by pushing them towards the inside of a term.

47 Expansion theorem

Parallel composition can be eliminated equationally by expanding its possible initial moves. Suppose

$$P = \sum_{i \in I} \alpha_i.P_i, \quad Q = \sum_{j \in J} \beta_j.Q_j.$$

Then the expansion law has the form

$$P \mid Q = \sum_{i \in I} \alpha_i.(P_i \mid Q) + \sum_{j \in J} \beta_j.(P \mid Q_j) + \sum_{\substack{i \in I, j \in J \\ \alpha_i = \beta_j}} \tau.(P_i \mid Q_j).$$

The first two groups represent independent moves; the last group represents synchronizations.

Esempio Expansion with synchronization

Let

$$p = \alpha.p' + \beta.p'', \quad q = \bar{\alpha}.q' + \gamma.q''.$$

The expansion of $p \mid q$ contains the independent moves $\alpha, \beta, \bar{\alpha}, \gamma$, and the synchronization summand

$$\tau.(p' \mid q').$$

If channel α is then restricted, the external α - and $\bar{\alpha}$ -moves disappear while the internal τ -move remains possible.

48 Soundness for strong bisimilarity

Teorema Soundness

The axioms for Basic CCS, restriction, relabelling, and expansion are sound for strong bisimilarity. Therefore,

$$E_{CCS} \vdash P = Q \implies P \sim Q.$$

The proof has two levels. First, check that both sides of every axiom are strongly bisimilar. Then use the fact that $\sim$ is an equivalence relation and a congruence: every rule of equational logic therefore preserves bisimilarity.

49 Completeness for finite CCS

49.1 Standard form

Definizione Standard form

A finite process is in *standard form* if it has the shape

$$\sum_{i=1}^m \mu_i.P_i$$

and every derivative P_i is itself in standard form, recursively down to nil.

Using the axioms for restriction, relabelling, and expansion, every recursion-free CCS term can be transformed equationally into standard form.

Define the maximum depth $d(P)$ as the maximum length of a computation of the finite process P .

Teorema Completeness for strong bisimilarity

For finite CCS processes,

$$P \sim Q \implies E_{CCS} \vdash P = Q.$$

Proof Idea

Put both processes in standard form:

$$P = \sum_i \mu_i.P_i, \quad Q = \sum_j \nu_j.Q_j.$$

Proceed by induction on the maximum depth. By bisimilarity, every summand $\mu_i.P_i$ of P must be matched by a summand $\mu_i.Q_j$ of Q such that

$$P_i \sim Q_j.$$

By the induction hypothesis,

$$E_{CCS} \vdash P_i = Q_j.$$

Thus every summand of P is equationally represented in Q . The symmetric argument covers the summands of Q , and idempotence *A4* removes duplicates.

50 Weak equivalences and observational congruence

Weak bisimilarity $\approx$ is not a congruence for $+$. For a compositional equational theory we therefore consider the greatest congruence contained in $\approx$, called *observational congruence*.

Definizione Observational congruence

Two processes p, q are observationally congruent, written here

$$p \simeq_c q,$$

if every immediate transition

$$p \xrightarrow{\mu} p'$$

can be matched by the appropriate weak transition of q , respecting the initial observation, to a derivative q' satisfying

$$p' \approx q',$$

and symmetrically for transitions of q .

The distinction from weak bisimulation is especially important for an initial τ -transition: observational

congruence does not allow it to be discarded arbitrarily inside a choice context. We have

$$\sim \subseteq \simeq_c \subseteq \approx.$$

Teorema Hennessy's lemma

If $p \approx q$, then at least one of the following holds:

$$p \simeq_c q, \quad \tau.p \simeq_c q, \quad p \simeq_c \tau.q.$$

51 Tau laws

The weak axiomatization adds the laws

$$\begin{aligned} \tau 1 \quad & \mu.\tau.P = \mu.P, \\ \tau 2 \quad & P + \tau.P = \tau.P, \\ \tau 3 \quad & \mu.(P + \tau.Q) = \mu.(P + \tau.Q) + \mu.Q. \end{aligned}$$

These equations saturate a term with respect to weakly reachable behaviour.

51.1 Full standard form

A standard form is *full* if it explicitly contains all summands corresponding to the relevant weak transitions: if

$$P \xRightarrow{\mu} P',$$

then the form contains, up to the equational theory, the appropriate direct summand $\mu.P'$.

Proposition Saturation lemma

If P is in standard form and

$$P \xRightarrow{\mu} P',$$

then the axioms $\tau 1, \tau 2, \tau 3$ allow us to derive

$$P = P + \mu.P'$$

in the form required by the observational semantics.

Proof

Proceed by induction on the depth of P . Since P is in standard form and $P \xRightarrow{\mu} P'$, consider the following cases.

1. If $\mu.P'$ is already a summand of P , then

$$P = P + \mu.P'$$

follows directly from A4.

2. Suppose $\mu.Q$ is a summand of P and

$$Q \xRightarrow{\tau} P'.$$

By the induction hypothesis,

$$EQ_\tau \vdash Q = Q + \tau.P'.$$

Hence

$$\begin{aligned}
P &= P + \mu.Q & (A4) \\
&= P + \mu.(Q + \tau.P') & (EQ_\tau \vdash Q = Q + \tau.P') \\
&= P + \mu.(Q + \tau.P') + \mu.P' & (\tau 3) \\
&= P + \mu.Q + \mu.P' & (EQ_\tau \vdash Q = Q + \tau.P') \\
&= P + \mu.P' & (A4).
\end{aligned}$$

3. Suppose $\tau.Q$ is a summand of P and

$$Q \stackrel{\mu}{\Rightarrow} P'.$$

By the induction hypothesis $EQ_\tau \vdash Q = Q + \mu.P'$, and

$$\begin{aligned}
P &= P + \tau.Q & (A4) \\
&= P + \tau.Q + Q & (\tau 2) \\
&= P + \tau.Q + Q + \mu.P' & (EQ_\tau \vdash Q = Q + \mu.P') \\
&= P + \tau.Q + \mu.P' & (\tau 2) \\
&= P + \mu.P' & (A4).
\end{aligned}$$

Every standard form can therefore be transformed into a full standard form without changing the relevant depth.

Proof Reduction to full standard form

Proceed by induction on the depth of P . At depth zero, $P \equiv \text{nil}$. Otherwise, recursively transform every continuation of the summands of P into full standard form without increasing its depth. Then consider all pairs (μ_i, P_i) such that

$$P \stackrel{\mu_i}{\Rightarrow} P_i \quad \text{but} \quad P \not\stackrel{\mu_i}{\Rightarrow} P_i.$$

By the saturation lemma, all missing summands can be added equationally:

$$EQ_\tau \vdash P = P + \mu_1.P_1 + \cdots + \mu_k.P_k.$$

The right-hand side is in full standard form and has the same depth as P .

Teorema Weak completeness

The axiomatization obtained by adding the tau laws is sound and complete for observational congruence on finite processes.

Proof Idea

Put two observationally congruent terms in full standard form and proceed by induction on depth. Hennessy's lemma reduces the comparison of weakly bisimilar derivatives to three observational-congruence cases, possibly prefixing one derivative by τ . The tau laws and the induction hypothesis then match all summands of the two forms; associativity, commutativity, and idempotence conclude the equational equality.

More precisely, if $\mu.P'$ is a summand of P , observational congruence and the full standard form of Q yield a summand $\mu.Q'$ with $P' \approx Q'$. By Hennessy's lemma there are three cases:

1. $P' \simeq_c Q'$: the induction hypothesis gives directly $EQ_\tau \vdash P' = Q'$;
2. $P' \simeq_c \tau.Q'$: reduce $\tau.Q'$ to full standard form, apply induction, and then use $\tau 1$ to derive $EQ_\tau \vdash$

$$\mu.P' = \mu.Q';$$

3. $\tau.P' \simeq_c Q'$: symmetric to the previous case.

Repeat the argument for every summand of Q ; A4 removes duplicates.

Part IX

Modal logics and model checking

52 Equivalence checking and model checking

Let $Impl$ be an implementation of a reactive system, for example a process written in CCS. There are two basic approaches to the verification of its correctness.

In *equivalence checking* we compare the implementation with a complete behavioural specification:

$$Impl \equiv Spec,$$

where $\equiv$ is an abstract behavioural equivalence, for example strong or weak bisimilarity. The specification is usually expressed in the same language as the implementation and describes the whole intended behaviour.

In *model checking* we verify one particular property:

$$Impl \models F.$$

Here $\models$ is a satisfaction relation and F is a property, usually expressed by a logic. Thus a logical formula gives a partial specification of the intended behaviour.

52.1 Modal and temporal properties

A modal property describes what a system can or must do from its current state. A temporal property describes how a property behaves along the possible evolutions of the system. Typical properties of reactive systems include:

- *safety*: something bad never happens;
- *liveness*: something good eventually happens;
- *possibility*: a state satisfying a property can be reached;
- *inevitability*: all reachable states satisfy a property.

53 Hennessy–Milner Logic

53.1 Syntax

Definizione Hennessy–Milner Logic

Let Act be the set of actions of an LTS. The formulae of Hennessy–Milner Logic are generated by

$$F, G ::= tt \mid ff \mid F \wedge G \mid F \vee G \mid \langle a \rangle F \mid [a]F, \quad a \in Act.$$

The modality $\langle a \rangle F$ is a *possibility*: the process can perform an a -action and reach a process satisfying F . The modality $[a]F$ is a *necessity*: every a -derivative, if any, satisfies F .

In particular,

$$\langle a \rangle tt$$

expresses the capability of performing an a -action, whereas

$$[a]ff$$

expresses the inability to perform an a -action. For example,

$$\langle \text{coin} \rangle (\langle \text{coffee} \rangle \text{tt} \vee \langle \text{tea} \rangle \text{tt})$$

requires the possibility of inserting a coin and then obtaining either coffee or tea.

For a finite set $A = \{a_1, \dots, a_n\} \subseteq \text{Act}$, we use the abbreviations

$$\langle A \rangle F \triangleq \bigvee_{i=1}^n \langle a_i \rangle F, \quad [A]F \triangleq \bigwedge_{i=1}^n [a_i]F.$$

53.2 Satisfaction relation

Definizione Semantics of HML

Let

$$(\text{Proc}, \text{Act}, \{\xrightarrow{a}\}_{a \in \text{Act}})$$

be an LTS. The satisfaction relation $p \models F$ is defined by

1. $p \models \text{tt}$ for every $p \in \text{Proc}$;
2. $p \not\models \text{ff}$ for every $p \in \text{Proc}$;
3. $p \models F \wedge G$ if and only if $p \models F$ and $p \models G$;
4. $p \models F \vee G$ if and only if $p \models F$ or $p \models G$;
5. $p \models \langle a \rangle F$ if and only if

$$\exists p' \in \text{Proc} \mid p \xrightarrow{a} p' \wedge p' \models F;$$

6. $p \models [a]F$ if and only if

$$\forall p' \in \text{Proc}, \quad p \xrightarrow{a} p' \implies p' \models F.$$

Notice the vacuity in the semantics of the box modality. If p has no a -derivative, then $p \models [a]F$ for every F . Consequently,

$$\langle a \rangle \text{ff} \equiv \text{ff}, \quad [a] \text{tt} \equiv \text{tt},$$

and

$$p \models [a] \text{ff} \iff p \not\xrightarrow{a}.$$

Esempio Elementary formulae

The following formulae have the direct readings

$$\begin{aligned} E \models \langle \text{tick} \rangle \text{tt} &\iff E \text{ can perform a tick,} \\ E \models \langle \text{tick} \rangle \langle \text{tock} \rangle \text{tt} &\iff E \text{ can perform a tick and then a tock,} \\ E \models \langle \{ \text{tick}, \text{tock} \} \rangle \text{tt} &\iff E \text{ can perform a tick or a tock,} \\ E \models [\text{tick}] \text{ff} &\iff E \text{ cannot perform a tick.} \end{aligned}$$

Esempio Tick and tock

Consider the recursive process

$$C_1 = \text{tick}.C_1.$$

Then

$$C_1 \models [\text{tick}] (\langle \text{tick} \rangle \text{tt} \wedge [\text{tock}] \text{ff}).$$

After every tick, the process can perform another tick and cannot perform a tock.

53.3 Negation

Negation is not a primitive operator of HML, but it can be defined by duality.

Definizione Complement of a HML formula

Define recursively

$$\begin{aligned} \text{tt}^c &= \text{ff}, & \text{ff}^c &= \text{tt}, \\ (F \wedge G)^c &= F^c \vee G^c, & (F \vee G)^c &= F^c \wedge G^c, \\ (\langle a \rangle F)^c &= [a]F^c, & ([a]F)^c &= \langle a \rangle F^c. \end{aligned}$$

Teorema

For every HML formula F and process p ,

$$p \models F^c \iff p \not\models F.$$

Proof

The proof is by structural induction on F . The Boolean cases follow from De Morgan laws. For the modal case,

$$p \not\models \langle a \rangle F$$

means that every a -derivative of p fails to satisfy F , equivalently every a -derivative satisfies F^c . Hence

$$p \models [a]F^c.$$

The case of $[a]F$ is dual.

54 Model checking HML

For a finite LTS, satisfaction of a HML formula can be checked bottom-up by labelling states with the subformulae they satisfy. We first determine the states satisfying the innermost subformulae, then propagate the labels through Boolean connectives and modalities.

Esempio Choice and modal formulae

Consider

$$P = a.0 + a.b.0.$$

Then

$$P \models \langle a \rangle \langle b \rangle \text{tt},$$

$$P \models \langle a \rangle [b] \text{ff},$$

$$P \not\models [a] \langle b \rangle \text{tt},$$

$$P \not\models [a] [b] \text{ff}.$$

The first two formulae are existential with respect to the initial a -choice, so it is enough to select a suitable derivative. The last two are universal with respect to all a -derivatives and fail because the two branches have different behaviours.

Esercizio

Given a finite LTS and a HML formula, compute the satisfaction set by labelling the states bottom-up from the atomic formulae to the whole formula.

55 HML and strong bisimilarity

Modal formulae are able to observe branching structure, not only traces. In particular, they can distinguish when a nondeterministic choice is resolved.

Esempio The position of a choice

We have

$$\begin{aligned} a.(b.0 + c.0) &\models \langle a \rangle (\langle b \rangle \text{tt} \wedge \langle c \rangle \text{tt}), \\ a.b.0 + a.c.0 &\not\models \langle a \rangle (\langle b \rangle \text{tt} \wedge \langle c \rangle \text{tt}). \end{aligned}$$

Similarly,

$$\begin{aligned} a.b.0 &\models [a] \langle b \rangle \text{tt}, \\ a.b.0 + a.0 &\not\models [a] \langle b \rangle \text{tt}, \end{aligned}$$

and

$$\begin{aligned} a.b.(c.0 + d.0) &\models [a] \langle b \rangle \langle c \rangle \text{tt}, \\ a.b.c.0 + a.b.d.0 &\not\models [a] \langle b \rangle \langle c \rangle \text{tt}. \end{aligned}$$

Finally,

$$a.(b.c.0 + b.d.0) \models [a] (\langle b \rangle \langle c \rangle \text{tt} \wedge \langle b \rangle \langle d \rangle \text{tt}),$$

whereas

$$a.b.c.0 + a.b.d.0 \not\models [a] (\langle b \rangle \langle c \rangle \text{tt} \wedge \langle b \rangle \langle d \rangle \text{tt}).$$

Teorema Hennessy–Milner theorem

Let p, q be processes.

1. If $p \sim q$, then for every HML formula F ,

$$p \models F \iff q \models F.$$

2. If the LTS is image finite and p, q satisfy exactly the same HML formulae, then

$$p \sim q.$$

Proof

For the first implication, proceed by structural induction on F . The Boolean cases are immediate. Suppose

$$p \models \langle a \rangle F.$$

Then there exists p' with

$$p \xrightarrow{a} p' \quad \text{and} \quad p' \models F.$$

Since $p \sim q$, there exists q' such that

$$q \xrightarrow{a} q' \quad \text{and} \quad p' \sim q'.$$

By the inductive hypothesis $q' \models F$, hence $q \models \langle a \rangle F$. The converse direction is symmetric, and the box modality is analogous.

For the second implication, define

$$R = \{(s, t) \mid \forall F, s \models F \iff t \models F\}.$$

We show that R is a bisimulation. Suppose $s R t$ and

$$s \xrightarrow{a} s'.$$

Assume by contradiction that no a -derivative of t is related by R to s' . Since the LTS is image finite, write the finitely many a -derivatives of t as

$$t_1, \dots, t_n.$$

For each i , since $s' \not/R t_i$, there exists a formula F_i distinguishing them. Replacing F_i by its complement if necessary, we can assume

$$s' \models F_i, \quad t_i \not\models F_i.$$

Therefore

$$s \models \langle a \rangle \bigwedge_{i=1}^n F_i,$$

while t does not satisfy this formula, contradicting sRt . The symmetric condition is identical.

Definizione Image-finite LTS

An LTS is *image finite* if for every process p and action a , the set

$$\{p' \mid p \xrightarrow{a} p'\}$$

is finite.

Thus, on image-finite systems, strong bisimilarity is exactly logical equivalence with respect to HML.

56 Modal depth

A finite HML formula can inspect a process only up to a finite depth.

Definizione Modal depth

The modal depth of an HML formula is defined recursively by

$$\begin{aligned} \text{md}(\text{tt}) &= \text{md}(\text{ff}) = 0, \\ \text{md}(F \wedge G) &= \text{md}(F \vee G) = \max\{\text{md}(F), \text{md}(G)\}, \\ \text{md}(\langle a \rangle F) &= \text{md}([a]F) = \text{md}(F) + 1. \end{aligned}$$

Teorema

Let F be a HML formula and let $k = \text{md}(F)$. If the defender has a defending strategy in the strong bisimulation game between s and t for k rounds, then

$$s \models F \iff t \models F.$$

Consequently, there is no finite HML formula that detects a deadlock at an arbitrarily large unknown depth in a general LTS: the deadlock may occur after a trace longer than the modal depth of the formula. This motivates the introduction of recursive formulae.

57 Temporal properties and recursion

57.1 Inevitability and possibility

Consider the two properties

$$\begin{aligned} s \models \text{Inev}(F) &\iff \text{all states reachable from } s \text{ satisfy } F, \\ s \models \text{Poss}(F) &\iff \text{some state reachable from } s \text{ satisfies } F. \end{aligned}$$

Assume that

$$\text{Act} = \{a_1, \dots, a_n\}$$

is finite and define

$$\langle \text{Act} \rangle F \triangleq \bigvee_{i=1}^n \langle a_i \rangle F, \quad [\text{Act}] F \triangleq \bigwedge_{i=1}^n [a_i] F.$$

Formally, one would like to write

$$\text{Inev}(F) \equiv F \wedge [\text{Act}] F \wedge [\text{Act}][\text{Act}] F \wedge \dots$$

and

$$\text{Poss}(F) \equiv F \vee \langle \text{Act} \rangle F \vee \langle \text{Act} \rangle \langle \text{Act} \rangle F \vee \dots$$

These are infinite formulae and therefore do not belong to ordinary HML. The same properties can instead be described by the recursive equations

$$X = F \wedge [\text{Act}] X$$

and

$$X = F \vee \langle \text{Act} \rangle X.$$

57.2 Recursive formulae

Definizione HML with one recursively defined variable

Extend HML with a variable X :

$$F ::= X \mid \text{tt} \mid \text{ff} \mid F_1 \wedge F_2 \mid F_1 \vee F_2 \mid \langle a \rangle F \mid [a] F.$$

The variable is equipped with either a least or a greatest fixed-point definition

$$X \stackrel{\min}{=} F_X \quad \text{or} \quad X \stackrel{\max}{=} F_X,$$

where F_X may contain occurrences of X .

The distinction between least and greatest solutions is necessary because recursive equations may have no solution, one solution, or several solutions.

Esempio Recursive equations may have different numbers of solutions

Over the natural numbers,

$n = 2n$	has the unique solution $n = 0$,
$n = n + 1$	has no solution,
$n = n$	has every natural number as a solution.

Over sets of integers,

$M = \{7\} \cap M$	has the solutions $\emptyset$ and $\{7\}$,
$M = \mathbb{Z} \setminus M$	has no solution,
$M = \{3\} \cup M$	has every $M \supseteq \{3\}$ as a solution.

For a recursive HML equation such as

$$X = [a]\text{ff} \vee \langle a \rangle X,$$

we therefore need a principled choice among the possible sets of processes satisfying the equation. Fixed-point theory provides exactly this choice.

58 Set-theoretic semantics of HML

Instead of asking whether a single process satisfies a formula, we associate with each formula the set of all processes that satisfy it.

Definizione Denotational semantics of HML

Let F be a non-recursive HML formula. Define

$$\begin{aligned} \llbracket \text{tt} \rrbracket &= \text{Proc}, \\ \llbracket \text{ff} \rrbracket &= \emptyset, \\ \llbracket F \wedge G \rrbracket &= \llbracket F \rrbracket \cap \llbracket G \rrbracket, \\ \llbracket F \vee G \rrbracket &= \llbracket F \rrbracket \cup \llbracket G \rrbracket. \end{aligned}$$

For $S \subseteq \text{Proc}$, define the modal operators

$$\begin{aligned} \langle\langle a \rangle\rangle S &\triangleq \{p \in \text{Proc} \mid \exists p' \in S, p \xrightarrow{a} p'\}, \\ \llbracket a \rrbracket S &\triangleq \{p \in \text{Proc} \mid \forall p', p \xrightarrow{a} p' \implies p' \in S\}. \end{aligned}$$

Then

$$\llbracket \langle a \rangle F \rrbracket = \langle\langle a \rangle\rangle \llbracket F \rrbracket, \quad \llbracket [a] F \rrbracket = \llbracket a \rrbracket \llbracket F \rrbracket.$$

Teorema

For every process p and non-recursive HML formula F ,

$$p \models F \iff p \in \llbracket F \rrbracket.$$

Proof

Immediate by structural induction on F .

Esempio Modal operators on sets

For a suitable finite LTS, if

$$S = \{1, 4\},$$

the modal operators can yield, for example,

$$\langle\langle a \rangle\rangle S = \{0, 3\}, \quad \llbracket a \rrbracket S = \{1, 2, 3, 4\}.$$

The first set contains the states having at least one a -successor in S ; the second contains the states whose every a -successor belongs to S , including states with no a -successor.

59 Functionals and fixed-point semantics

A formula F_X containing a free variable X induces an operator on sets of processes.

Definizione Functional associated with a recursive formula

For $S \subseteq \text{Proc}$, define $O_{F_X}(S)$ recursively as the denotation of F_X when the variable X is interpreted by S . In particular,

$$O_X(S) = S.$$

Boolean and modal operators are interpreted exactly as in the previous section.

All the constructors of the language are monotone with respect to set inclusion. Therefore the operator associated with a formula in which the variable occurs positively is monotone:

$$S \subseteq T \implies O_{F_X}(S) \subseteq O_{F_X}(T).$$

Since

$$(\mathcal{P}(\text{Proc}), \subseteq)$$

is a complete lattice, Tarski's theorem guarantees both a least and a greatest fixed point.

Definizione Semantics of a recursive variable

If

$$X \stackrel{\min}{=} F_X,$$

then

$$\llbracket X \rrbracket = \text{lfp}(O_{F_X}).$$

If

$$X \stackrel{\max}{=} F_X,$$

then

$$\llbracket X \rrbracket = \text{gfp}(O_{F_X}).$$

Equivalently, for a monotone $f: \mathcal{P}(\text{Proc}) \rightarrow \mathcal{P}(\text{Proc})$,

$$\text{lfp}(f) = \bigcap \{S \subseteq \text{Proc} \mid f(S) \subseteq S\},$$

whereas

$$\text{gfp}(f) = \bigcup \{S \subseteq \text{Proc} \mid S \subseteq f(S)\}.$$

For a finite set of processes we can compute the fixed points by iteration:

$$\emptyset, f(\emptyset), f^2(\emptyset), \dots$$

for the least fixed point and

$$\text{Proc}, f(\text{Proc}), f^2(\text{Proc}), \dots$$

for the greatest fixed point.

Esercizio

Determine whether the following functions are monotone and compute their fixed points when they exist:

$$f(X) = X \cup \{s, t\}, \quad g(X) = \text{Proc} \setminus X.$$

60 Examples of recursive properties

60.1 Reaching a state without an action

Consider

$$X \stackrel{\min}{=} [a]\text{ff} \vee \langle \text{Act} \rangle X.$$

The formula says that a state can reach, in finitely many transitions, a state that cannot perform an a -action. For one finite LTS, possible fixed points include

$$\{0, 2\}, \quad \{0, 1, 2\},$$

and least-fixed-point iteration gives

$$\emptyset \subseteq \{2\} \subseteq \{0, 2\}.$$

Thus the least solution is $\{0, 2\}$. For a system in which no state can reach in finitely many steps a state without an a -transition, the least fixed point is $\emptyset$.

60.2 An invariant capability

Consider

$$X \stackrel{\max}{=} \langle a \rangle \text{tt} \wedge [\text{Act}] X.$$

A state satisfies this property if it can perform an a -action and all its derivatives satisfy the same property. In finite examples the greatest solution can be, for instance,

$$\{0, 1\}$$

or, in another LTS,

$$\{1\}.$$

60.3 Reaching a state that can perform b

The least fixed point

$$X \stackrel{\min}{=} \langle b \rangle \text{tt} \vee \langle \{a, b\} \rangle X$$

expresses that there exists a finite path of a - or b -actions leading to a state from which a b -transition is available. In the corresponding finite example the solution is

$$\{0, 1, 2, 4\}.$$

60.4 Always being able to continue with b

Consider

$$X \stackrel{\max}{=} \langle b \rangle \text{tt} \wedge [b] X.$$

A state satisfies the property if it has at least one b -transition and every b -derivative satisfies the property again. In the finite example the greatest fixed point is

$$\{s_1, s_2, t_1\}.$$

61 A selection of temporal properties

The following recursive schemes are useful for expressing common temporal properties.

Definizione Invariance and possibility

$$\begin{aligned}\text{Inv}(F) &\triangleq \max X.(F \wedge [\text{Act}]X), \\ \text{Pos}(F) &\triangleq \min X.(F \vee \langle \text{Act} \rangle X).\end{aligned}$$

Definizione Safety and eventually

$$\begin{aligned}\text{Safe}(F) &\triangleq \max X.(F \wedge ([\text{Act}]\text{ff} \vee \langle \text{Act} \rangle X)), \\ \text{Even}(F) &\triangleq \min X.(F \vee (\langle \text{Act} \rangle \text{tt} \wedge [\text{Act}]X)).\end{aligned}$$

The Even property requires F to be reached after finitely many steps along every evolution that continues: before reaching F , at least one transition must remain available and all possible next states must satisfy the same recursive requirement.

Definizione Weak and strong until

Weak until and strong until are defined by

$$\begin{aligned}F U^w G &\triangleq \max X.(G \vee (F \wedge [\text{Act}]X)), \\ F U^s G &\triangleq \min X.(G \vee (F \wedge \langle \text{Act} \rangle \text{tt} \wedge [\text{Act}]X)).\end{aligned}$$

For weak until, G is not required to occur: F may remain true forever. For strong until, G must eventually be reached along all possible evolutions. In particular,

$$\text{Inv}(F) \equiv F U^w \text{ff}, \quad \text{Even}(F) \equiv \text{tt} U^s F.$$

62 Nested and mutually recursive formulae

Recursive definitions can be nested. For example,

$$X \stackrel{\min}{=} Y \vee \langle \text{Act} \rangle X, \quad Y \stackrel{\max}{=} \langle a \rangle \text{tt} \wedge \langle \text{Act} \rangle Y.$$

Since the definition of Y does not depend on X , we first compute $\llbracket Y \rrbracket$ and then $\llbracket X \rrbracket$.

Variables may also be mutually recursive, for example

$$X \stackrel{\max}{=} [a]Y, \quad Y \stackrel{\max}{=} \langle a \rangle X.$$

In this case the semantic domain is the product lattice

$$\mathcal{P}(\text{Proc}) \times \mathcal{P}(\text{Proc})$$

with componentwise order

$$(S_1, S_2) \subseteq (S'_1, S'_2) \iff S_1 \subseteq S'_1 \wedge S_2 \subseteq S'_2.$$

The two equations induce one monotone functional on the product lattice and are solved simultaneously by its greatest fixed point.

Teorema Characteristic property for finite-state processes

Let s be a process with finitely many reachable states. There exists a recursive property X_s such that, for every process t ,

$$s \sim t \iff t \in \llbracket X_s \rrbracket.$$

The characteristic formula follows the transition structure of s : diamond modalities require matching transitions of s , box modalities exclude unmatched transitions of the compared process, and recursive variables represent derivatives of s .

63 Bisimilarity as a greatest fixed point

The fixed-point view of recursive logic mirrors the fixed-point view of bisimilarity itself. Let

$$F_{\sim} : \mathcal{P}(\text{Proc} \times \text{Proc}) \rightarrow \mathcal{P}(\text{Proc} \times \text{Proc})$$

be defined by putting $(p, q) \in F_{\sim}(R)$ when every transition of p can be matched by q with derivatives related by R , and symmetrically every transition of q can be matched by p .

A relation R is a bisimulation exactly when

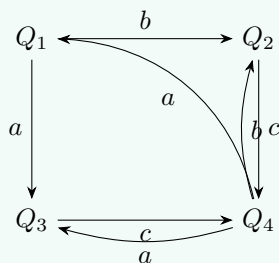
$$R \subseteq F_{\sim}(R).$$

Therefore

$$\sim = \text{gfp}(F_{\sim}).$$

Esempio Iteration of the bisimulation functional

Consider the LTS



corresponding to

$$\begin{aligned} Q_1 &= b.Q_2 + a.Q_3, \\ Q_2 &= c.Q_4, \\ Q_3 &= c.Q_4, \\ Q_4 &= b.Q_2 + a.Q_3 + a.Q_1. \end{aligned}$$

Starting from the total relation,

$$R_0 = \{Q_1, Q_2, Q_3, Q_4\}^2,$$

one application of the functional leaves

$$R_1 = \text{Id} \cup \{(Q_1, Q_4), (Q_4, Q_1), (Q_2, Q_3), (Q_3, Q_2)\}.$$

A second application removes the pairs (Q_1, Q_4) and (Q_4, Q_1) :

$$R_2 = \text{Id} \cup \{(Q_2, Q_3), (Q_3, Q_2)\}.$$

The next application is stable, so

$$R_3 = R_2 = \text{gfp}(F_{\sim}).$$

The same distinction appears in the bisimulation game. From (Q_4, Q_1) , the attacker can choose the extra transition

$$Q_4 \xrightarrow{a} Q_1.$$

Matching it with $Q_1 \xrightarrow{a} Q_3$ leads to (Q_1, Q_3) , where the available actions do not match.

64 Main references

Useful references include:

- K. R. Apt and E.-R. Olderog, *Verification of Sequential and Concurrent Programs*;
- W. Fokkink, *Introduction to Process Algebra*;
- R. Milner, *Communication and Concurrency*;
- A. W. Roscoe, *The Theory and Practice of Concurrency*;
- H. Bowman and R. Gomez, *Concurrency Theory*;
- J. C. M. Baeten and W. P. Weijland, *Process Algebra*;
- M. Hennessy, *Algebraic Theory of Processes*;
- R. J. van Glabbeek, work on the linear-time–branching-time spectrum;
- D. Sangiorgi and D. Walker, *The π -Calculus: A Theory of Mobile Processes*.